Publication Number: JP 7874373 B1
PUBLISHED ON 16-Jun-2026

Application Number: JP 2026-23007 A
FILED ON 16-Feb-2026

Ｗｅｂアプリケーション操作支援システム、方法、およびプログラム (Japanese)

Web application operation support system, method, and program (English)

Japan Invention Patent (Source: IFI)

Inventors: 井浦知久

Applicants: かなでき株式会社

Attorney, Agent or Firm: 弁理士法人Toreru

Examiners: Primary: 松平 英

Classifications: International (2018.01, invention): G06F 9/451
FI: G06F 9/451

Priority: JP 2026-23007 A (16-Feb-2026)

Backward Patent Citations:
(Cited by: EX = examiner)
CN 115080046 A 一种页面设计中多组件抽象关联融合方法和装置 (20-Sep-2022) | EX
CN 116010280 A 一种基于自然语言的自动化测试方法、装置、设备及介质 (25-Apr-2023) | EX
JP 2004-5676 A 双方主導マルチ・モーダル対話及び関連ブラウジング機構を作成するための方法及びシステム (8-Jan-2004) | EX
JP 2021-12728 A メディア環境内におけるインテリジェント自動アシスタント (4-Feb-2021) | EX
JP H06-230716 A 手話通訳装置および方法 (19-Aug-1994) | EX
(continued)

Abstract (English)

[Problem] To provide a web application operation support system, method, and program that can continue to operate even after a change in UI structure. [Solution] The method includes the steps of: acquiring the structure and operable elements of each page of a target web application; describing each page as structured metadata consisting of multiple categories including subject, operation type, target area, constraints, and context; extracting and associating semantically identifiable features for each operable element in addition to location information; saving the metadata in a referable format; and enabling continuous operation even after a change in UI structure by having a processing means operating in the user's browser environment convert the user's natural language input into the same format as the structured metadata, and when identifying an operable element by referring to the metadata, searching for similar elements using semantically identifiable features if the search using location information fails. [Selected Figure] Figure 2

Abstract (Japanese)

[Image Omitted]

【課題】ＵＩ構造変更後も継続的に動作可能なＷｅｂアプリケーション操作支援システム、方法及びプログラムを提供する。
【解決手段】方法は、対象Ｗｅｂアプリケーションの各ページの構造及び操作可能要素を取得し、各ページを主体、操作種別、対象領域、制約条件及びコンテキストを含む複数のカテゴリからなる構造化メタデータとして記述するステップと、各操作可能要素について、位置情報に加えて、当該要素を意味的に識別可能な特徴を抽出して関連付けるステップと、メタデータを参照可能な形式で保存するステップと、ユーザーのブラウザ環境において動作する処理手段が、ユーザーの自然言語入力を構造化メタデータと同一形式に変換し、メタデータを参照して操作対象要素を特定する際、位置情報による検索が失敗した場合に、意味的に識別可能な特徴を用いて類似要素を探索することで、ＵＩ構造変更後も継続的に動作可能とするステップと、を含む。
【選択図】図２

Backward Patent Citations (continuation)

US 2022/0164540 A1 Analyzing Underspecified Natural Language Utterances in a Data Visualization User Interface (26-May-2022) | EX
JP 06230716 A

Synopsis AI Document Summarization (Default)

Summary Size: Default

Key Phrases

- web application
- target web application
- web application operation support
- natural language

Patent Document 4 describes a technology for using post-event analysis of dialogue logs to improve performance, and is different from a system that autonomously performs GUI operations on a web application during operation. Furthermore, it does not disclose a configuration that identifies the target of operation based on metadata that semantically describes pages and elements, rather than DOM location information (CSS selectors, XPate, etc.).

Thus, conventional technologies could not achieve self-healing capabilities that would allow for continuous operation even when minor UI changes (such as selector changes or element position changes) occurred, while simultaneously enabling GUI operations via natural language instructions without modifying existing web applications.

Therefore, the present invention aims to provide a web application operation support system, method, and program that can continuously operate by adding natural language or voice operation functions via scripts (widgets) embedded within the application's pages without modifying the existing web application, and by rediscovering the target element using semantic features even after changes to the UI structure.

To solve the above problems, the present invention provides a Web application operation support service provision method that adds natural language operation functionality to an existing Web application without modifying it, and is characterized by including the steps of: acquiring the structure and operable elements of each page of the target Web application; describing each page as structured metadata consisting of multiple categories including subject, operation type, target area, constraints, and context; extracting and associating semantically identifiable features for each operable element in addition to location information; accumulating and managing the metadata in a database on a tenant basis; issuing a script corresponding to the tenant and embedding it in the pages of the target Web application to deploy an agent that operates in the user's browser; and enabling continuous operation even after UI structure changes by enabling the agent to convert the user's natural language input into the same format as the structured metadata and, when identifying an operable element by referring to the metadata, searching for similar elements using the semantically identifiable features if the search by location information fails.

Furthermore, the present invention is characterized by a gap-driven decision engine that stores an input representation containing multiple slots extracted from the user's natural language input, and a state representation consisting of multiple slots of the same format representing the currently displayed web application page or operation state. It detects differences or omissions (hereinafter referred to as "gaps") between these two representations and determines the next action to be taken according to the type and combination of the gap. This decision engine operates based on a definition that declaratively associates the gap patterns with corresponding actions, without relying on procedural logic that sequentially describes conditional branching. This allows for consistent determination of whether to perform execution, screen transition, question, suggest candidates, or request confirmation, even when the utterance is incomplete or ambiguous.

This figure shows an example of the overall system configuration of the Web application operation support service according to this embodiment.This flowchart shows an example of the processing flow from user natural language input to operation execution according to this embodiment.This figure shows an example of a fallback search control mechanism for identifying DOM elements according to this embodiment.This figure shows an example of a mechanism for saving and reusing successful operation patterns according to this embodiment.

<First Embodiment>
The Web application operation support service provision method 100 according to this embodiment is a service provision method that adds natural language operation functionality to an existing Web application 20 without modifying it, and includes the steps of: obtaining the structure and operable elements of each page of the target Web application 20, describing each page as structured metadata 62 consisting of multiple categories including subject, operation type, target area, constraints, and context; extracting and associating semantically identifiable features 92 with each operable element in addition to location information 91; and data of the metadata 62 on a tenant basis. The present invention is characterized by including the steps of: storing and managing data in a base 60; issuing a script 40 corresponding

to the tenant and embedding it in the page of the target web application 20 to deploy an agent 50 that operates in the user 30's browser; and enabling continuous operation even after a change in the UI structure by enabling the agent 50 to convert the user 30's natural language input into the same format as the structured metadata 62, and when the agent 50 identifies the target element by referring to the metadata 62, if the search using location information 91 fails, searching for similar elements using the semantically identifiable features 92.

Furthermore, the Web application operation support service provision method 100 preferably includes the following steps: estimating the operations that can be performed on each page from the acquired metadata 62 and element information; automatically generating test scenarios written in natural language for each estimated operation (step 260); saving the generated test scenarios in association with the metadata 62; and automatically updating the test scenarios by comparing existing test scenarios with newly acquired results (step 261) upon reacquisition. This method allows for the automatic generation of operation tests for the target Web application 20 in natural language format from the acquired results without manual test case definition.

The decision-making method 120 according to this embodiment is a decision-making method for an agent 50 that operates an existing web application 20 using natural language, and is characterized by including the steps of: analyzing the utterance of a user 30 into an utterance expression 270 of a predetermined format consisting of a plurality of slots 272; storing a page or operation of the web application 20 as a definition expression 271 having the same slot structure 272 as the predetermined format; projecting the utterance expression 270 and the definition expression 271 into the same slot space 273 and comparing them, and detecting missing slots 274 in the utterance expression 270 or gaps 275 between the two expressions; filling in the detected missing slots or gaps 276 to generate a completed utterance expression; and determining the next action for the web application 20 based on the completed utterance expression.

Furthermore, the correspondence between the gap patterns and action types is defined by a declarative correspondence table defined as an external parameter, and can be adjusted without modifying the program itself. This configuration allows for flexible modification of the judgment criteria according to the characteristics and operational requirements of the target web application, while eliminating complex conditional branching and improving maintainability and scalability.

The self-healing operation support system 10 comprises a semantic analysis means that converts the user's natural language utterance into five-slot structured data (Actor, Operation, Scope, Constructs, Context), a page description means that stores each page of the target web application as metadata describing it with the same five-slot structure, a determination means that detects the gap between the utterance slot and the current page slot and determines the next action, and a control means that switches between executing DOM operations, presenting candidate UIs, answering questions, and displaying confirmation dialogs according to the determination result. This system prioritizes searching using location information on the DOM (CSS selector, XPh), and if the search fails, it uses semantic features (display text, role attribute, neighbor label, relative position, etc.) saved during crawling to search for similar elements. It also includes a self-healing element selection means that rediscovers the most similar element as the target of operation using weighted scoring. Even if the UI structure of the target application changes, it can rediscover the target of operation using semantic features without script modification and continue to operate.

The target web application 20 is an existing web application that this system supports. A wide variety of web applications are targeted, including business systems, administration screens, and public service sites, regardless of whether authentication is required. By simply embedding a single line of widget tag (script tag) into the target web application 20, voice and natural language operation functionality can be added retrofitted without any modification to the application itself. A crawler autonomously discovers the entire structure of the target web application from a single seed URL, describes each page as 5-slot structured data, extracts and saves the semantic features (PCI) of the elements, and collects knowledge information such as help text and tooltips. This allows the system to understand the structure, function, and terminology of the target web application 20, appropriately interpret the user's natural language utterances, and perform operations on their behalf.

<Example of workflow definition>
In the RPA workflow design screen, it can be defined as follows:

Step 1: Retrieve metadata (first time only)
- TargetURL: https://example. com/admin
- OutputVariable: siteMetadata

Step 2: Launch browser (standard RPA function)
-Browser: Chrome
- URL: https://example. com/admin

Step 3: Natural Language Web Manipulation (Activity of the present invention)
- InputText: Display list of responsible persons - Metadata: siteMetadata

-Browser: currentBrowser

Step 4: Natural Language Web Manipulation (Activity of the present invention)
- InputText: Added Taro Tanaka - Metadata: siteMetadata
-Browser: currentBrowser

This embodiment allows for the addition of natural language operation functionality without modifying the existing web application 20, and enables the realization of a web application operation support system 10 that can continue to operate even after UI structure changes. Furthermore, by learning successful patterns, the response speed of frequently occurring operations is improved, providing users 30 with a smoother operation experience.

<Appearance 1>
A method for enabling natural language operation without modifying an existing web application 20, comprising the steps of: acquiring the structure and operable elements of each page of the target web application 20, describing each page as structured metadata 62 consisting of multiple categories including subject, operation type, target area, constraints, and context; extracting and associating semantically identifiable features 92 with each operable element in addition to location information 91; saving the metadata 62 in a referable format; and enabling continuous operation even after a change in the UI structure by having an agent 50 operating in the user 30's browser environment convert the user 30's natural language input into the same format as the structured metadata 62, and when identifying an operable element by referring to the metadata 62, searching for similar elements using semantically identifiable features 92 if the search using location information 91 fails.

(Variant)
The vectorization method is not limited to this embodiment, and other embedding models or hybrid similarity measures can be used. Furthermore, the similarity threshold can be dynamically adjusted and automatically learned based on the user's operation history and environment settings.

100 Web application operation support service provision method 110 Test automation method 120 Decision-making method 10 Self-healing operation support system 20 Target Web application 30 User 40 Script 41 Tenant-specific identifier 50 Agent 60 Tenant-unit metadata management database 61 Tenant identifier 62 Structured metadata 63 Operation subject category 64 Operation type category 65 Target area category 66 Constraint category 67 Context category 70 Input conversion unit 80 Metadata storage unit 90 Element information storage unit 91 Location information 92 Semantic identification features 150 Judgment unit 160 Element selection unit 161 Fallback search control 162 Weighted scoring 163 Candidate presentation mechanism 164 User confirmation switching mechanism 170 Execution unit 171 Operation execution mechanism 172 Explanation response mechanism 180 UI change detection mechanism 181 Differential update mechanism 190 Acquisition process 191 Navigation area analysis 192 Link extraction 193 Action button detection 194 Dynamic form recording 195 Help text extraction 196 Tooltip extraction 200 Authentication processing 201 Temporary receipt of authentication information 202 Automatic login identification 203 Browser session maintenance patrol 210 Control policy 211 Risk classification 212 Risk level 213 Confirmation required 214 Correction grace period 215 Correction grace period 216 Grace period management 220 Conversation history 221 Context maintenance mechanism 230 Operation target inference mechanism 231 Operation history 232 Inference source identifier 233 Confirmation processing 240 Language model 250 Operation pattern database 251 Confidence level update 260 Automatic test scenario generation 261 Test scenario update 262 Test scenario result recording 270 Utterance representation 271 Definition representation 272 Slot structure 273 Slot space projection 274 Missing slot detection 275 Gap detection 276 Completion

Document Description

This invention relates to a web application operation support system, method, and program.

In recent years, with the advancement of digital transformation, the web-based transformation of business operations within companies and organizations has accelerated. Web applications are being used in a wide variety of fields, including SaaS applications, e-commerce sites, government electronic application systems, and medical information systems, and there is a growing need for increased efficiency and automation in operating these systems. In particular, with the development of natural language processing and artificial intelligence technologies, there is a growing expectation that users will be able to operate these systems using natural language and voice.

Against this backdrop, technological development related to conversational agents and chatbots is progressing. Patent Document 1 discloses a chatbot system for automating crew support tasks. This system uses natural language processing technology to understand user inquiries and provide appropriate answers, and improves conversational quality by learning from user feedback using machine learning algorithms.

Patent Document 2 discloses a system that uses a chatbot generation API to generate answers to questions and automates communication between stores and call centers. This system discloses the development of a proprietary AI that uses manuals as training data to generate response text in accordance with the manuals.

Patent Document 3 discloses an AI system for automatically evaluating and improving the performance of a virtual dialogue agent. This system automatically generates a ground truth from a knowledge source, simulates natural language dialogue interaction using a simulator to evaluate performance, and if the performance does not meet a threshold, a remediation manager identifies and implements improvement actions.

Patent Document 4 discloses a conversation log analysis system that receives conversation log files related to interactions with an automated virtual dialogue agent in a dialogue system, syntactically parses the dialogue events to extract behavioral characteristics from the traces, clusters each trace based on its proximity to result values such as success or failure, and searches for symmetric patterns by applying pattern specifications with time conditions, thereby utilizing the results to improve the performance of the virtual dialogue agent.

Japanese Patent Publication No. 2025-057380Japanese Patent Publication No. 2025-055706Special Publication No. 2024-506519Special Publication No. 2023-510241

However, Patent Documents 1 and 2 primarily focus on generating responses to user inquiries and providing dialogue support, and do not describe technologies for autonomously executing GUI operations (screen transitions, input, clicks, etc.) based on user natural language instructions for existing web applications. In particular, they do not disclose a configuration for identifying DOM elements of a target web application based on semantic features, without relying on location information such as CSS selectors or XPate.

Furthermore, in recent years, tools such as UiPath have provided AI-based element recognition capabilities in the field of RPA (Robotic Process Automation). While these tools identify the target of operation through image recognition and DOM analysis, they rely on pre-recorded operation flows and do not perform autonomous decision-making based on a semantic understanding of the application structure.

Patent Document 3 describes a system focused on performance evaluation and improvement of the virtual dialogue agent itself, and does not perform GUI operations on actual web applications. Furthermore, it does not disclose DOM structure analysis, semantic identification of UI elements, or self-healing functions for UI changes.

Furthermore, browser automation frameworks such as Selenium and Playwright rely on location-based element identification using CSS selectors and XPate, and while they have fallback mechanisms using multiple selectors, these merely enumerate different ways of specifying the same element and lack the self-healing capability to rediscover elements based on semantic characteristics after UI structure changes.

Patent Document 4 describes a technology for using post-event analysis of dialogue logs to improve performance, and is different from a system that autonomously performs GUI operations on a web application during operation. Furthermore, it does not disclose a configuration that identifies the target of operation based on metadata that semantically describes pages and elements, rather than DOM location information (CSS selectors, XPate, etc.).

Thus, conventional technologies could not achieve self-healing capabilities that would allow for continuous operation even when minor UI changes (such as selector changes or element position changes) occurred, while simultaneously enabling GUI operations via natural language instructions without modifying existing web applications.

Therefore, the present invention aims to provide a web application operation support system, method, and program that can continuously operate by adding natural language or voice operation functions via scripts (widgets) embedded within the application's pages without modifying the existing web application, and by rediscovering the target element using semantic features even after changes to the UI structure.

To solve the above problems, the present invention provides a Web application operation support service provision method that adds natural language operation functionality to an existing Web application without modifying it, and is characterized by including the steps of: acquiring the structure and operable elements of each page of the target Web application; describing each page as structured metadata consisting of multiple categories including subject, operation type, target area, constraints, and context; extracting and associating semantically identifiable features for each operable element in addition to location information; accumulating and managing the metadata in a database on a tenant basis; issuing a script corresponding to the tenant and embedding it in the pages of the target Web application to deploy an agent that operates in the user's browser; and enabling continuous operation even after UI structure changes by enabling the agent to convert the user's natural language input into the same format as the structured metadata and, when identifying an operable element by referring to the metadata, searching for similar elements using the semantically identifiable features if the search by location information fails.

Furthermore, the present invention is characterized by a gap-driven decision engine that stores an input representation containing multiple slots extracted from the user's natural language input, and a state representation consisting of multiple slots of the same format representing the currently displayed web application page or operation state. It detects differences or omissions (hereinafter referred to as "gaps") between these two representations and determines the next action to be taken according to the type and combination of the gap. This decision engine operates based on a definition that declaratively associates the gap patterns with corresponding actions, without relying on procedural logic that sequentially describes conditional branching. This allows for consistent determination of whether to perform execution, screen transition, question, suggest candidates, or request confirmation, even when the utterance is incomplete or ambiguous.

According to this invention, natural language or voice control functionality can be added to existing web applications simply by embedding scripts (widgets, Widget tags) within the page, without modifying the existing web application. Furthermore, by identifying the target of operation based on the semantic characteristics of elements, independent of location information on the DOM, continuous operation becomes possible through self-correction even after UI structure changes. This reduces the maintenance burden during operation and enables operation support that can be universally applied to different applications. In particular, while conventional RPA and browser automation tools are limited to fragile fallbacks based on multiple location specifications, this invention achieves essential element rediscovery based on the semantic characteristics of elements (display text, role attributes, proximity relationships, etc.), providing true self-correction against UI changes.

This figure shows an example of the overall system configuration of the Web application operation support service according to this embodiment.This flowchart shows an example of the processing flow from user natural language input to operation execution according to this embodiment.This figure shows an example of a fallback search control mechanism for identifying DOM elements according to this embodiment.This figure shows an example of a mechanism for saving and reusing successful operation patterns according to this embodiment.

The following description of this embodiment will be made with reference to the drawings. It should be noted that the present invention is not limited to the embodiments shown below, and can be modified, added, modified, or deleted to the extent that a person skilled in the art can conceive of it. Any embodiment that achieves the functions and effects of the present invention is included within the scope of the present invention. The Web application operation support system, method, and program according to the present invention are not merely abstract information processing or the presentation of rules, but rather work in cooperation with the hardware resources constituting the computer to realize concrete information processing. Specifically, the present invention is executed on an information processing device equipped with a processor, memory, display device, input device, communication interface, etc. The processor executes the program to acquire, analyze, monitor updates to, and generate operation commands for the DOM structure via a Web browser, and outputs these operation commands to the target Web application via the input device or communication interface. Furthermore, the present invention acquires changes in the screen state displayed on the display device, user operations from the input device, and the state of the execution environment which depends on hardware resources such as the DOM tree and event queue managed by the browser. These are reflected in the structured metadata and determination processing to control the identification of target elements and the execution of operations. Furthermore, the self-healing element selection and operation execution control in this invention works in conjunction with a browser engine and display control mechanism operating on hardware to detect physical state changes in the execution environment, such as screen transitions, element

rearrangement, or changes in DOM structure, and determines the next operation based on the results. Thus, this invention realizes concrete and executable operation control for web applications through the cooperation of hardware resources and software processing constituting a computer, and possesses a technical concept as a software-related invention.

Furthermore, the system of the present invention has multiple embodiments regarding its arrangement. Representative arrangements are described below.

<First deployment method: SaaS-type script embedding>
This configuration involves embedding a single line of script tag within the target web application's page, allowing the agent to operate within the user's browser. This configuration is suitable for providing the service as a SaaS because it requires minimal modification to the target application. The section describing embodiments for implementing the present invention will primarily explain this configuration.

<Second deployment method: Browser extension>
This is implemented as a browser extension, such as Chrome Extension or Firefox Add-on. In this form, no code addition to the target web application is required; users can use it by installing the extension in their browser. Each part of the agent (input conversion unit, judgment unit, element selection unit, etc.) operates as a content script for the browser extension.

<Third Deployment Configuration: RPA Platform Integration>
This implementation integrates as a function within existing RPA tools (such as UiPath, Blue Prism, and Automation Anywhere). In this configuration, it works in conjunction with the RPA platform's browser control function, providing semantic analysis using HASP, element selection using Semantic Selector, and judgment functionality through slot gap completion. In the RPA workflow definition, "Web operations using natural language" can be used as a single activity.

<Fourth deployment method: Desktop agent>
It operates as a standalone desktop application, controlling the browser via the browser's DevTools Protocol (CDP) or WebDriver. This configuration is suitable as an enterprise solution for unified operation of multiple web applications within a company.

<Common elements>
In all of the above configurations, the following core technologies are common:

(1) Semantic page description using HASP (5-slot structured metadata) (2) Self-healing element selection based on semantic features (3) Judgment engine using slot gap completion (4) Flash RAG (real-time ranking of DOM-derived candidates)

The deployment configuration should be appropriately selected based on the target environment, security requirements, integration with existing systems, and other factors.

<First Embodiment>
The Web application operation support service provision method 100 according to this embodiment is a service provision method that adds natural language operation functionality to an existing Web application 20 without modifying it, and includes the steps of: obtaining the structure and operable elements of each page of the target Web application 20, describing each page as structured metadata 62 consisting of multiple categories including subject, operation type, target area, constraints, and context; extracting and associating semantically identifiable features 92 with each operable element in addition to location information 91; and data of the metadata 62 on a tenant basis. The present invention is characterized by including the steps of: storing and managing data in a base 60; issuing a script 40 corresponding to the tenant and embedding it in the page of the target web application 20 to deploy an agent 50 that operates in the user 30's browser; and enabling continuous operation even after a change in the UI structure by enabling the agent 50 to convert the user 30's natural language input into the same format as the structured metadata 62, and when the agent 50 identifies the target element by referring to the metadata 62, if the search using location information 91 fails, searching for similar elements using the semantically identifiable features 92.

Furthermore, it is preferable that the aforementioned multiple categories consist of five categories: operating entity 63 (Actor), operation type 64 (Operation), target area 65 (Scope), constraints 66 (Constraints), and context 67 (Context).

Furthermore, the acquisition step preferably includes steps 191 and 192 of analyzing the navigation area within the seed URL page and extracting the linked destinations within that area as priority targets; steps 193 and 194 of detecting action buttons on each page, detecting and recording the forms that dynamically expand when those buttons are clicked; and steps 195 and 196 of extracting the help text, tooltips, and explanatory text contained on each page and saving them in association with the metadata 62.

Furthermore, the acquisition step preferably further includes steps 201 and 202 of temporarily receiving authentication information from the Web application 20 requiring authentication at the start of acquisition, automatically identifying the login form and completing authentication; a step of discarding the authentication information from memory after authentication is complete and not permanently storing it; and a step 203 of traversing all pages while maintaining the same browser session.

Furthermore, it is preferable that the semantically identifiable feature 92 includes the displayed text, element attribute information, relationship to neighboring elements, and relative position within the form.

Furthermore, it is preferable that the script 40 includes a tenant-specific identifier 41, and that the agent 50 uses the identifier 41 upon startup to retrieve the latest metadata 62 corresponding to the tenant from the server, and performs a differential update by re-retrieving the metadata if it detects a UI change in the target application 20.

Furthermore, it is preferable to further include the steps of defining a control policy 210 as an external parameter, which includes an operation risk classification 211, whether or not pre-execution verification is required 213, and a correction grace period 214, and distributing and applying the control policy 210 to the agent 50, thereby reserving the final execution authority for update operations to the user 30.

Furthermore, it is preferable that the application of the control policy 210 includes the steps of: classifying the interpreted operations into risk levels 212; requiring explicit confirmation for high-risk operations and automatically executing medium and low-risk operations after a correction grace period 215; setting the length of the correction grace period 215 variably based on the risk level 212 of the operation and the reliability of the analysis; and resetting the grace period if additional input is detected during the correction grace period 215, canceling execution if an interruption instruction is detected, and automatically executing the operation if the grace period has elapsed without interruption.

The self-healing operation support system 10 according to this embodiment is a system that enables operation by natural language or voice without modifying the application itself, by using a script 40 embedded in the pages of an existing Web application 20, and operates within the user 30's browser, and includes an input conversion unit 70 that converts the user 30's natural language input into structured data consisting of multiple categories including subject, operation type, target area, constraints, and context; a metadata storage unit 80 that acquires and stores metadata 62 describing each page of the target Web application 20 in the multiple categories from the server based on a tenant identifier 41; and for each operable element of each page, The system comprises: an element information holding unit 90 that holds location information 91 along with semantically identifiable features 92 for the element; a determination unit 150 that detects differences between the structured data derived from the input and the metadata 62 of the current page and determines the next action; an element selection unit 160 that attempts to search for the target element using location information 91, and if the search fails, searches for and selects a similar element using the semantically identifiable features 92; and an execution unit 170 that executes the operation based on the determination unit 150. It is characterized by the ability to rediscover the target element using the semantically identifiable features 92 and continue operation without script modification, even if the UI structure of the target application 20 changes.

Furthermore, the element selection unit 160 preferably performs the following fallback operations in this order: searching by location information 91, searching by identifier attributes assigned to elements, and searching for similar elements by semantically identifiable features 92. It then calculates the similarity using weighted scoring 162 of the semantically identifiable features 92. If multiple candidates have equivalent scores, it is preferable to switch to user confirmation or candidate presentation depending on the risk level 212 of the operation.

Furthermore, the input conversion unit 70 analyzes user utterances as operation types, specifically data manipulation (create, retrieve, update, delete) and explanation requests (explain), using the same 5-slot structured data format. The determination unit 150 outputs a determination result indicating that a DOM operation should be performed for data manipulations, and a determination result indicating that knowledge information associated with the metadata storage unit 80 should be searched for and used as the answer for explanation requests. The system 10 preferably maintains the conversation history 220 in continuous dialogues where data manipulations and explanation requests are mixed, allowing for switching between operations and explanations while maintaining context.

Furthermore, it is preferable to further include means for saving the operation pattern in a database 250 if the operation analysis by the language model 240 is successful; means for searching the database 250 when new input is received and reusing the pattern without calling the language model 240 if a similar pattern exists with a confidence level above a predetermined threshold; and means 251 for recording the number of times the pattern has been used and the number of successes, and dynamically updating the confidence level.

Furthermore, the determination unit 150 includes means 230 for inferring the target of operation by referring to past conversation history 220 or operation history 231 when the target of operation is not explicitly stated from the user 30's input; means for setting an identifier 232 indicating the source of the inference for the inferred target of operation; and means 233 for confirming the inferred target of operation with the user 30 when the identifier 232 indicates an inference from the conversation history 220 and the operation type is a write operation. If the user 30 confirms, it is preferable to update the identifier 232 to an explicit specification and execute the operation, while for inputs where the target of operation is explicitly specified, the operation is executed immediately without confirmation.

The test automation method 110 according to this embodiment is a test automation method for a Web application 20 using the self-healing operation support system 10, and includes the steps of: converting test cases written in natural language into structured data using the input conversion unit 70; identifying target elements based on semantically identifiable features 92 rather than location information 91 using the element selection unit 160; and performing operations on the identified elements and recording the test results 262. The method is characterized in that, even after a UI change of the target Web application 20, testing can be continued without redefining the test cases by rediscovering elements based on the semantically identifiable features 92.

Furthermore, the Web application operation support service provision method 100 preferably includes the following steps: estimating the operations that can be performed on each page from the acquired metadata 62 and element information; automatically generating test scenarios written in natural language for each estimated operation (step 260); saving the generated test scenarios in association with the metadata 62; and automatically updating the test scenarios by comparing existing test scenarios with newly acquired results (step 261) upon reacquisition. This method allows for the automatic generation of operation tests for the target Web application 20 in natural language format from the acquired results without manual test case definition.

The program according to this embodiment is a program for causing the computer to function as a component of the self-healing operation support system 10.

The decision-making method 120 according to this embodiment is a decision-making method for an agent 50 that operates an existing web application 20 using natural language, and is characterized by including the steps of: analyzing the utterance of a user 30 into an utterance expression 270 of a predetermined format consisting of a plurality of slots 272; storing a page or operation of the web application 20 as a definition expression 271 having the same slot structure 272 as the predetermined format; projecting the utterance expression 270 and the definition expression 271 into the same slot space 273 and comparing them, and detecting missing slots 274 in the utterance expression 270 or gaps 275 between the two expressions; filling in the detected missing slots or gaps 276 to generate a completed utterance expression; and determining the next action for the web application 20 based on the completed utterance expression.

The Web application operation support service provision method 100 according to this embodiment is a SaaS-type service provision method that enables interactive operation using natural language or voice without modifying existing Web applications. In the crawl phase, it autonomously discovers the structure of the target Web application and generates metadata describing each page as 5-slot structured data (Actor, Operation, Scope, Constructs, Context). In the runtime phase, it converts the user's natural language utterance into the same 5-slot structure, determines the next action using a gap-driven determination engine, identifies the target of operation through self-healing element selection based on semantic features, and executes DOM operations. This method achieves resilience to changes in the UI structure and self-healing capabilities by identifying pages and elements by "meaning" rather than "location" on the DOM.

Test automation method 110 is a test automation method that executes test cases written in natural language while self-correcting in response to UI changes using a self-healing element selection technology based on semantic features. This method converts test cases (operation procedures) written in natural language into five-slot structured data, identifies target elements based on semantic features rather than CSS selectors, performs DOM operations on the identified elements, and records the test results. Even after UI changes to the target web application (selector changes, element position changes, etc.), the test can continue to run without redefining the test cases by rediscovering the target elements based on semantic features. This eliminates vulnerabilities in conventional tests that rely on CSS selectors or XPath, reduces maintenance costs, and allows business personnel to define tests.

<Configuration and operating principle of a gap-driven decision engine>
The determination unit 150 according to this embodiment represents the user input analyzed by HASP as an input slot consisting of multiple slots including subject, operation type, target area, constraints, and context, and simultaneously holds the current page or operation state as a state slot consisting of the same slot structure.
The determination unit 150 compares the input slot and the state slot in the same semantic space and detects a gap between them by determining whether each slot matches, does not match, or is missing.

The determination unit 150 classifies the detected gap combinations as predetermined gap patterns and determines the next process to be executed based on the action type associated with the gap pattern. The action type includes at least operation execution, screen transition, question request, candidate presentation, and confirmation request.

Furthermore, the correspondence between the gap patterns and action types is defined by a declarative correspondence table defined as an external parameter, and can be adjusted without modifying the program itself. This configuration allows for flexible modification of the judgment criteria according to the characteristics and operational requirements of the target web application, while eliminating complex conditional branching and improving maintainability and scalability.

Decision-making method 120 is a declarative slot gap determination method that determines the next action based on user input. This method structures user input as an input slot containing the actor, operation type, and scope. It maintains the current state as a state slot of the same format and compares the input slots with the state slots to determine the match, mismatch, or missing information for each slot. The gap combinations are classified into multiple patterns (insufficient information, scope match, scope mismatch, actor type mismatch, operation only specified, etc.). The response actions corresponding to each pattern (execution, question, candidate presentation, confirmation request, answer) are mapped as a declarative pattern-to-action table, rather than using conditional branching logic, to determine the next action. The pattern definitions, mappings, and determination thresholds are declaratively defined as external parameter files, allowing the determination logic to be replaced without modifying the algorithm itself.

The self-healing operation support system 10 comprises a semantic analysis means that converts the user's natural language utterance into five-slot structured data (Actor, Operation, Scope, Constructs, Context), a page description means that stores each page of the target web application as metadata describing it with the same five-slot structure, a determination means that detects the gap between the utterance slot and the current page slot and determines the next action, and a control means that switches between executing DOM operations, presenting candidate UIs, answering questions, and displaying confirmation dialogs according to the determination result. This system prioritizes searching using location information on the DOM (CSS selector, XPh), and if the search fails, it uses semantic features (display text, role attribute, neighbor label, relative position, etc.) saved during crawling to search for similar elements. It also includes a self-healing element selection means that rediscovers the most similar element as the target of operation using weighted scoring. Even if the UI structure of the target application changes, it can rediscover the target of operation using semantic features without script modification and continue to operate.

The target web application 20 is an existing web application that this system supports. A wide variety of web applications are targeted, including business systems, administration screens, and public service sites, regardless of whether authentication is required. By simply embedding a single line of widget tag (script tag) into the target web application 20, voice and natural language operation functionality can be added retrofitted without any modification to the application itself. A crawler autonomously discovers the entire structure of the target web application from a single seed URL, describes each page as 5-slot structured data, extracts and saves the semantic features (PCI) of the elements, and collects knowledge information such as help text and tooltips. This allows the system to understand the structure, function, and terminology of the target web application 20, appropriately interpret the user's natural language utterances, and perform operations on their behalf.

User 30 is a user who operates the target web application 20 using voice or natural language. A diverse range of users are anticipated, including users unfamiliar with web application operation, visually impaired individuals, the elderly, and workers with both hands occupied. User 30 can complete operations of the business system without complex mouse and keyboard operations by giving verbal instructions for screen operation through a semi-transparent dedicated screen (AURA) or by inputting text in natural language. User 30's utterances are automatically executed after a correction grace period, enabling smooth, continuous operation. High-risk operations require explicit confirmation, preventing errors. Furthermore, the same architecture can handle not only operation requests but also requests for explanations such as "What is XX?", allowing users to search a knowledge base and receive answers, including those specific to the target application, via voice.

Script 40 is a single-line Widget tag (JavaScript script) embedded in the target web application 20. This script 40 is issued by the crawler via the dashboard, and the company representative embeds it into the HTML of the target web application 20 to enable voice and natural language interaction functionality. Script 40 contains a tenant-specific identifier 41, which is used by the agent 50 to read metadata corresponding to the target web application 20 from the tenant-specific metadata management database 60 during execution. With this script 40, the target web application 20 becomes voice and natural language interactive without modification, a Widget (semi-transparent UI) is displayed on the screen, and it is ready to receive user 30's input.

The tenant-specific identifier 41 is an identifier included in the script 40 that uniquely identifies the target web application 20. This identifier 41 is generated when the crawler registers the site URL on the dashboard and is embedded in the script tag. During execution, the agent 50

uses this identifier 41 to search the tenant-specific metadata management database 60 for the corresponding tenant identifier 61 and reads the structured metadata 62, route information, page information, knowledge, success patterns, etc., associated with that tenant. This allows for accurate separation and management of metadata for each tenant, even when multiple target web applications 20 are provided as SaaS, enabling appropriate operational support.

Agent 50 is an AI agent that interprets the user's utterances during the runtime phase and operates the target web application 20. Agent 50 consists of multiple components, including an input conversion unit 70, a metadata storage unit 80, an element information storage unit 90, a determination unit 150, an element selection unit 160, an execution unit 170, a UI change detection mechanism 180, an acquisition process 190, an authentication process 200, a control policy 210, a conversation history 220, an operation target prediction mechanism 230, an operation pattern database 250, and an automatic test scenario generation 260. Agent 50 receives voice or text input from user 30, structures it using HASP (5-slot parsing), determines the next action (EXECUTE, SUGGEST, ASK, CONFIRM, ANSWER) using Gap Model, semantically identifies elements using Semantic Selector, generates an ActionPlan to execute DOM operations, and responds to user 30 with the operation results via voice or text. Agent 50 reduces LLM calls and speeds up response times by 30 to 50 times through Flash RAG and success pattern caching.

The tenant-specific metadata management database 60 is a database that centrally manages metadata for each target web application 20. This database 60 stores structured metadata 62, route information, page information, knowledge, test cases, success patterns, RAG vector data, etc., using the tenant identifier 61 as the key. The structured metadata 62 includes HASP metadata describing each page as 5-slot structured data, element semantic characteristics (PCI), navigation structure, dynamic form information, help text, tooltips, etc. During execution, the agent 50 uses the tenant-specific identifier 41 to read the corresponding metadata from this database 60, interprets the user's utterance 30, and uses it to perform appropriate operations. During recrawl, the existing metadata is compared with the new crawl results, and the latest state is maintained through differential updates.

The tenant identifier 61 is a key that uniquely identifies the target web application 20 in the tenant-specific metadata management database 60. This identifier 61 corresponds to the tenant-specific identifier 41 generated when the crawler registers the site URL on the dashboard and embedded in the script 40. During execution, the agent 50 receives the tenant-specific identifier 41 and searches for the metadata corresponding to this identifier 61. This identifier 61 allows for the separation of metadata for multiple tenants within the database, enabling accurate management of each tenant's structured metadata 62, route information, page information, knowledge, success patterns, etc., and enabling simultaneous support of multiple target web applications 20 as a SaaS-type service.

The structured metadata 62 is metadata that describes each page of the target web application 20 as 5-slot structured data. This metadata 62 is generated when a crawler crawls each page and extracts natural language summaries and structured slots according to HASP slots (Actor, Operation, Scope, Constraints, Context). Specifically, it includes an operator category 63, an operation type category 64, a target area category 65, a constraint category 66, and a context category 67. At runtime, the agent 50 holds this metadata 62 as the current page state slot and uses it to detect gaps with input slots extracted from the user 30's utterances and determine the next action. Structured metadata 62 is generated for all pages discovered during crawling and stored in the tenant-specific metadata management database 60.

The operator category 63 is a category corresponding to the Actor slot in the structured metadata 62, and describes "who" and "what" is being operated on. For example, it defines the types of entities handled by the target web application 20, such as "person in charge," "user," "product," and "facility." Each entity is assigned a type such as person, organization, or product as actor.type, and this is used in the judgment logic to compare the actor.type extracted from the user 30's utterance with the actor.type of the current page and switch to "SUGGEST" if there is a mismatch. This category 63 allows the system to recognize "Tanaka-san" as a person when the user 30 says "Add Tanaka-san," and to present candidate screens for registering people such as "person in charge" or "user."

The operation type category 64 corresponds to the Operation slot in the structured metadata 62 and describes the intent (CRUD) of the business operation. Specifically, it defines create (add, register), retrieve (search, list display), update (edit, update), delete (delete), and explain (request explanation). Each operation type is assigned a risk level and classified into high risk (delete, external transmission, settlement, permission change, etc.), medium risk (create, update), and low risk (retrieve, navigate, explain). Based on the operation type category 64, the determination unit 150 requests explicit confirmation (CONFIRM) for high-risk operations, automatically executes medium-risk operations after a correction grace period, and applies immediate execution or a short grace period for low-risk operations. This category 64 enables smooth, continuous operations while preventing errors.

The target area category 65 is a category corresponding to the Scope slot in the structured metadata 62, and describes "on which page" and "in which area" the operation is performed. For example, it defines each page or functional area of the target web application 20, such as "Staff List," "User Registration," "Product Management," and "Facility Details." The determination unit 150 compares the scope extracted from the user 30's utterance with the scope of the current page. If they match, it outputs "EXECUTE (Execute Operation)," and if they do not

match, it outputs "EXECUTE (Automatically Transition to the Relevant Page)." This category 65 allows the user 30 to automatically transition to the "User Registration" page and continue the operation even if the current page is the "Staff List" page when they utter "Add User."

The constraint category 66 corresponds to the Constraints slot in the structured metadata 62 and describes the conditions associated with the operation (number of items, order, filter, etc.). For example, it defines constraint conditions included in the user's utterance, such as "10 items," "by date," and "Tokyo only," or filter conditions that can be set on the page of the target web application 20. The execution unit 170 automatically performs actions such as inputting search conditions, changing the sort order, and applying filters based on the constraint category 66. This category 66 allows the system to set "Tokyo" as the filter condition, set "10 items" as the number of items to display, and execute a search on the relevant page when the user 30 utters, "Show 10 facilities in Tokyo."

The context category 67 corresponds to the Context slot in the structured metadata 62 and describes the conversation state and execution control state. Specifically, it includes the confidence level of the analysis, the source of scope inference (explicit specification/inference from conversation history), the presence or absence of correction intent (isCorrectionIntent), and the navigation history (navigationHistory). The determination unit 150 refers to the context category 67 and, if the confidence level is low, switches to a question or presents options (ASK /SUGGEST). If the scope source is "inference from conversation history" and it is a write operation, it requests confirmation processing (CONFIRM). If there is a correction intent, it cancels/corrects the previous operation. This category 67 enables appropriate control over ambiguous and context-dependent utterances, preventing erroneous operations.

The input conversion unit 70 is a processing unit that receives voice or text input from the user 30 and converts it into HASP (5-slot structured data). In the case of voice input, the speech recognition engine converts the voice to text, overwrites the text if additional utterances are made during the correction grace period, and finalizes the utterances once the correction grace period has elapsed. The finalized utterance text is structured into five slots—Actor, Operation, Scope, Constructs, and Context—through semantic analysis by a Large-Scale Language Model (LLM). If a success pattern cache exists, the process is accelerated by reusing similar patterns without calling the LLM. The input conversion unit 70 plays a central role in Hybrid Actionable Semantic Parsing, enabling the execution of judgment logic through the strictness of the structured slots while maintaining the flexibility of natural language.

The metadata storage unit 80 is a processing unit that stores structured metadata 62 read from the tenant-specific metadata management database 60 at runtime. This processing unit 80 stores the current page's HASP slot (state slot) in memory and references it to detect the gap between it and the input slot extracted by the determination unit 150 from the user's utterance. It also stores knowledge information, navigation structure, dynamic form information, etc., for each page, making them accessible to each component of the agent 50 as needed. When a page transition occurs, it reads the HASP slot of the destination page and updates the state slot. This processing unit 80 enables the agent 50 to accurately grasp the current context and perform appropriate decisions and operations.

The element information holding unit 90 is a processing unit that holds the semantic characteristics (PCI: Perceptual Content Identification) of each element extracted during crawling at runtime. This processing unit 90 holds the position information 91 (CSS selector, XPath, data attribute, etc.) and semantic identification characteristics 92 (display text, role attribute, aria-label, placeholder, neighbor label text, relative position within the form, etc.) of each element. The element selection unit 160 first searches for an element using the position information 91, and if unsuccessful, searches for a similar element using the semantic identification characteristics 92. This processing unit 90 enables self-correction by rediscovering the target based on semantic characteristics, even if the UI structure of the target web application 20 changes (selector changes, element position changes, etc.).

Location information 91 is information held in the element information holding unit 90 to identify the location of each element on the DOM. Specifically, it includes CSS selectors (e.g., #submit-btn, .user-name-input), XPat (e.g., //button[@id=submit-btn]), and data attributes assigned during crawling (e.g., data-adminal-id=submit-button). The element selection unit 160 first searches for the element using the location information 91, and if successful, immediately identifies it as the target for operation. If the search using location information 91 fails, the fallback search control 161 switches to a similar element search using semantic identification features 92. This information 91 enables fast and accurate element identification when the UI structure has not changed, and functions as a trigger for self-healing element selection when changes occur.

The semantic identification features 92 are fixed-format data stored in the element information holding unit 90 that describe the semantic features of each element. Specifically, they include display text (e.g., "Register", "Submit"), role attributes (e.g., button, textbox), aria-label (e.g., "Enter username"), placeholder (e.g., "Email address"), neighbor label text (e.g., text of the label element immediately preceding the input element), relative position within the form (e.g., "Third input field in the form"), etc. If the search using position information 91 fails, the element selection unit 160 extracts similar semantic features from elements currently on the DOM, calculates the similarity with the stored semantic identification features 92 using weighted scoring 162, and identifies the most similar element as the target for operation. This feature 92 mimics the process by which a human recognizes a button as a "register button" when viewing a screen, enabling self-healing element selection that does not rely on CSS selectors or XPath.

The determination unit 150 is a processing unit that detects the gap between the input slot extracted from the user's utterance 30 and the current page's state slot, and determines the next action (EXECUTE, SUGGEST, ASK, CONFIRM, ANSWER). This processing unit 150 implements a HASP Gap Model (gap-driven determination engine), determines the match/mismatch/missing status of each slot, classifies the gap combinations into multiple patterns (all three slots missing, scope matches the current page, scope mismatches, actor.type mismatches, only operation specified, etc.), and determines the response action according to a declarative table. The determination logic is defined as an external parameter file, allowing the determination rules to be replaced without changing the algorithm itself. This processing unit 150 eliminates complex conditional branching, allows for adjustment of determination parameters for each site, and improves maintainability through declarative definition.

The element selection unit 160 is a processing unit that identifies the element to be operated on when the determination unit 150 outputs "EXECUTE (operation execution)". This processing unit 160 includes a fallback search control 161, weighted scoring 162, a candidate presentation mechanism 163, and a user confirmation switching mechanism 164, realizing self-correcting element selection. First, it searches for elements using location information 91 (CSS selector, XPath). If this fails, it retries using the data attribute. If it fails again, it searches for similar elements using semantic identification features 92. The weighted scoring 162 selects the element if the candidate with the highest score exceeds a predetermined threshold. If multiple candidates have equivalent scores, it switches to user confirmation (CONFIRM) or candidate presentation (SUGGEST) depending on the risk level of the operation. This processing unit 160 enables self-correction by rediscovering the target element without script modification even if the UI structure of the target web application 20 changes.

The fallback search control 161 is a control mechanism in the element selection unit 160 that retries the element search using an alternative method if the search fails. This mechanism 161 executes the fallback process in the following order: (1) Search using the CSS selector or XPat of the location information 91; (2) If unsuccessful, retry using the data attribute (a semantic identifier assigned during crawling); (3) If still unsuccessful, search for similar elements using semantic identification features 92; (4) Based on a threshold determination, if the score is high, automatic selection occurs; if it is moderate, CONFIRM or SUGGEST occurs; and if it is low, ASK (re-selection) occurs. This mechanism 161 allows for gradual adaptation to changes in the UI structure, automatically rediscovering the target of the operation whenever possible, and, in cases of high uncertainty, requiring user confirmation or re-selection as a safety measure, thereby preventing erroneous operations while achieving self-correction.

The weighted scoring mechanism 162 is a mechanism in the element selection unit 160 that calculates a score for each candidate element when searching for similar elements using semantic identification features 92. This mechanism 162 calculates the score using the following weighting (example): text match × 0.40, role match × 0.25, context match (neighboring labels, etc.) × 0.25, position match (relative position) × 0.10. If the score is 0.8 or higher, automatic selection (AUTO) is performed; if it is between 0.5 and 0.8, confirmation or candidate suggestion (CONFIRM/SUGGEST) is performed; and if it is less than 0.5, re-selection (ASK) is performed. High-risk operations (delete, etc.) always require CONFIRM. This mechanism 162 combines multiple semantic features to select the most appropriate candidate, and applies safety measures when uncertainty is high, thereby achieving both self-healing and safety.

The candidate presentation mechanism 163 is a mechanism in the element selection unit 160 that presents candidates to the user 30 and prompts them to make a selection when multiple candidate elements have equivalent scores or when the scope is unclear. This mechanism 163 operates when the judgment unit 150 outputs "SUGGEST" and displays candidate pages or candidate operations in a ranked list. For example, if user 30 says "Add Tanaka-san," and actor.type is person, but the scope is unclear whether it is "person in charge" or "user," the mechanism presents options such as "Where do you want to add it? 1. Person in charge, 2. User." User 30 can then select "Select 2" by referring to the number, making a precise selection with a short token. This mechanism 163 allows for the presentation of appropriate candidates in response to ambiguous utterances, accurately understanding the user's intent, and preventing errors.

The user confirmation switching mechanism 164 is a mechanism in the element selection unit 160 that switches between automatic execution and user confirmation based on the risk level of the operation or the uncertainty of the score. This mechanism 164 operates according to the following rules: (1) High-risk operations (delete, external transmission, settlement, permission change, etc.) always require explicit confirmation (CONFIRM); (2) If the score is moderate (0.5 to 0.8), CONFIRM or SUGGEST is performed depending on the risk level of the operation; (3) If the score is low (less than 0.5), ASK (re-specification) is performed; (4) If the score is high (0.8 or higher) and the operation is moderate or low risk, automatic execution is performed after a correction grace period. This mechanism 164 enables smooth continuous operation while preventing erroneous operations, achieving both convenience and security for the user 30.

The execution unit 170 is a processing unit that executes actual DOM operations on the element identified by the element selection unit 160. This processing unit 170 includes an operation execution mechanism 171 and an explanation/response mechanism 172. The operation execution mechanism 171 executes DOM operations (click, text input, select, checkbox operation, page transition, etc.) according to the ActionPlan when the determination unit 150 outputs "EXECUTE" and the element selection unit 160 identifies the target of the operation. The explanation/response mechanism 172 searches the knowledge base when the determination unit 150 outputs "ANSWER," generates a response including a functional description specific to the target Web application 20, and responds to the user 30 in voice or text. After the

operation is completed, the success pattern is saved in the operation pattern database 250, reducing the number of LLM calls for similar utterances in the future.

The operation execution mechanism 171 is a mechanism that, in the execution unit 170, executes the actual DOM operation according to the ActionPlan when the determination unit 150 outputs "EXECUTE" and the element selection unit 160 identifies the target of the operation. This mechanism 171 automatically performs the following operations: button clicks (e.g., the "Register" button), text input (e.g., entering "Taro Tanaka" into the input element), select selections (e.g., selecting "Tokyo" from the dropdown), checkbox operations (e.g., turning on a checkbox), page transitions (e.g., clicking a navigation link to transition to the "User List" page), etc. After the operation is completed, the UI change detection mechanism 180 detects changes in the screen and updates the status slot of the metadata storage unit 80 as necessary. With this mechanism 171, the user 30 can complete operations of the business system using only natural language utterances, without performing complex mouse and keyboard operations.

The explanation response mechanism 172, when the execution unit 170 outputs "ANSWER" from the determination unit 150, searches the knowledge base, generates an answer including a functional explanation specific to the target web application 20, and responds to the user 30. This mechanism 172 operates when the user 30's utterance is an explain operation (e.g., "What is a care manager?", "How do I use this screen?"). It generates the answer by combining the help text extraction 195, tooltip extraction 196, and general knowledge from a large-scale language model extracted in the acquisition process 190. The answer is provided to the user 30 in voice or text, and an orb on the screen vibrates to visualize the progress of the reading. This mechanism 172 integrates operation support and explanation support in the same architecture, allowing the user 30 to immediately learn unfamiliar technical terms and operation methods while proceeding with the operation.

The UI change detection mechanism 180 is a processing mechanism that detects changes in the screen structure of the target web application 20 and updates metadata as needed. This mechanism 180 monitors screen changes such as page transitions, modal displays, and dynamic form deployments, and analyzes the differences in the DOM tree to identify the changes. The detected changes are reflected in the tenant-unit metadata management database 60 by the differential update mechanism 181, and only the changed parts are updated during the next crawl without re-analyzing the entire database. This mechanism 180 enables rapid response to minor changes in the target web application 20, maintains the freshness of metadata, and improves self-healing capabilities. Furthermore, for pages where changes occur frequently, it is used for adaptive control, such as increasing the priority of semantic feature search in the element selection unit 160.

The differential update mechanism 181 is a processing mechanism that reflects screen structure changes detected by the UI change detection mechanism 180 in the tenant-unit metadata management database 60. This mechanism 181 updates the element information (location information 91, semantic identification features 92) of the changed areas, re-analyzes the affected HASP slots (structured metadata 62), and synchronizes any additions or deletions to the knowledge information. Because it does not require a complete re-crawl, metadata updates are accelerated, allowing for rapid response to changes in the target web application 20 during operation. During re-crawling, the differentially updated metadata and the new crawl results are compared to verify consistency and correct as necessary. This mechanism 181 enables the maintenance of metadata freshness and continuous improvement of self-healing capabilities.

The acquisition process 190 is a series of processes performed during the crawling phase to extract semantic features, knowledge information, and structural information from each page of the target web application 20. This process 190 includes navigation area analysis 191, link extraction 192, action button detection 193, dynamic form recording 194, help text extraction 195, and tooltip extraction 196. Through these processes, the entire structure of the target web application is autonomously discovered from a single seed URL, each page is described as HASP structured data, semantic features (PCI) of elements are extracted and saved, and knowledge information is collected. The metadata generated by the acquisition process 190 is stored in the tenant-specific metadata management database 60 and referenced by the agent 50 during execution.

Navigation area analysis 191 is a process in the acquisition process 190 that identifies the navigation area (nav element, sidebar, header, menu, etc.) of each page and extracts the links within that area as priority crawl targets. This process 191 automatically identifies the navigation area using the semantic structure of the HTML (nav element, role=navigation, etc.) and CSS class names (e.g., .sidebar, .menu) as clues. Since links within the navigation area are often entry points to the main functional pages of the target web application 20, prioritizing their crawling allows for efficient discovery of the overall site structure. This process 191 enables autonomous crawling of the entire target web application from a single seed URL, eliminating the need to manually create sitemaps or URL lists.

Link extraction 192 is a process that extracts links (a elements, button elements, etc.) contained in each page during the acquisition process 190 and registers them as crawl targets. This process 192 extracts links in the page body, footer links, etc., in addition to the links prioritized and extracted in the navigation area analysis 191. For each link, the link text, href attribute, and estimated function of the linked page (e.g., "List of Personnel," "New Registration") are analyzed and recorded as a scope candidate for the HASP slot. Links to external sites, logout links, download links, etc., are excluded from crawling. This process 192 ensures that all pages of the target web application 20 are discovered without omission, estimates the function of each page, and collects basic information to generate structured metadata 62.

Action button detection 193 is a process in the acquisition process 190 that detects action buttons (new creation, edit, delete, save, etc.) contained on each page and records the screens (modals, inline forms, etc.) that are dynamically expanded when the corresponding button is clicked. This process 193 extracts button elements such as button elements, input [type=submit], and role=button, and estimates the operation type (create, update, delete, etc.) from the button's text content (e.g., "add new," "edit"). It detects the forms that are dynamically expanded when the button is clicked, records input elements, select elements, etc. within the form, and extracts the semantic characteristics of each field (label, placeholder, name attribute, etc.). This process 193 makes it possible to discover dynamic pages that would be missed by conventional crawlers and to comprehensively record all operable functions of the target web application 20.

The dynamic form recording 194 is a process that records the structure and semantic characteristics of each field of the dynamically expanding form (modal, inline form, slide-in form, etc.) detected by the action button detection 193 in the acquisition process 190. This process 194 extracts input elements, select elements, textarea elements, checkbox elements, etc., within the form, and records the text of the label element, placeholder, name attribute, id attribute, type attribute, required attribute, and nearby help text for each field. This information is used at runtime for the agent 50 to determine what to enter in which field. This process 194 enables field mapping from natural language utterances even for dynamically generated forms, allowing for accurate interpretation and execution of utterances such as "Enter Tanaka Taro for name."

The help text extraction process 195 extracts knowledge information such as help text, explanations, and warnings contained on each page during the acquisition process 190, and saves it in association with page metadata. This process 195 automatically identifies help text using specific CSS class names (e.g., .help-text, .description), role attributes (e.g., role=note), or placement patterns (e.g., small font text placed below a form field) as clues. The extracted help text includes functional descriptions, operating procedures, and definitions of technical terms for the target web application 20, and is used by the explanation response mechanism 172 to generate an answer when the user 30 utters an explain operation (explanation request). This process 195 automatically collects knowledge specific to the target web application 20, realizing a service that integrates operation support and explanation support.

The tooltip extraction process 196 extracts tooltips (title attribute, aria-label, custom tooltips, etc.) associated with each element during the acquisition process 190 and records them as semantic features of the element. This process 196 extracts the title attribute, aria-label attribute, data-tooltip attribute, etc., and also detects and records the content of tooltip elements that are dynamically displayed when the mouse hovers over them. Tooltips serve as important clues for understanding the meaning of elements, such as functional descriptions of buttons and links, and input examples for input fields. The extracted tooltips are stored as part of the semantic identification features 92 and are used by the element selection unit 160 when searching for similar elements, and by the explanation response mechanism 172 when generating answers. This process 196 collects knowledge information not explicitly displayed in the page text, enabling more accurate element identification and detailed explanation responses.

The authentication process 200 is a series of processes that, during the crawling phase, automatically logs in to the target web application 20 requiring authentication and continues crawling in an authenticated state. This process 200 includes temporary receipt of authentication information 201, automatic login identification 202, and browser session maintenance crawling 203. When a user 30 (company representative) enters authentication information (username, password, etc.) when registering a site URL on the dashboard, the crawler temporarily stores this information in memory, automatically identifies the login form, inputs and submits the information, and completes authentication. After authentication is complete, the authentication information is discarded from memory, and the authentication state is maintained by crawling all pages while maintaining the same browser session (BrowserContext). This process 200 enables crawling of web applications with authentication, allowing for the automatic collection of the overall structure and knowledge of the business system.

The temporary authentication information receipt process 201 is a process in the authentication process 200 that temporarily stores the authentication information (username, password, etc.) entered by the user 30 (company representative) when registering the site URL on the dashboard in memory. This process 201 encrypts the authentication information, stores it in memory, and retains it until the crawl process is complete. After the crawl is complete, or if authentication fails, the information is immediately discarded from memory and not stored in persistent storage (database, file, etc.). This process 201 enables crawling of authenticated web applications while ensuring security. While the user 30 needs to enter their authentication information each time a crawl is performed, the risk of the authentication information being leaked externally is minimized.

The automatic login identification 202 is a process in the authentication process 200 that automatically identifies the login form of the target web application 20, inputs and sends the authentication information held in the temporary authentication information receipt 201, and completes authentication. This process 202 recognizes the typical login form pattern (username field, password field, login button), automatically identifies the fields, and inputs their values. Field identification uses clues such as the name attribute (e.g., username, password), type attribute (e.g., text, password), and the text of the label element (e.g., "username", "password"). The login button is clicked to submit authentication and confirm successful authentication (e.g., transition to the dashboard page after login). This process 202 allows for automatic authentication completion and crawling of the page after authentication without manually defining the login procedure.

The browser session maintenance crawl 203 is a process performed during the authentication process 200 that crawls all pages of the target web application 20 while maintaining the same browser session (BrowserContext) even after authentication is complete. This process 203 uses the BrowserContext function of browser automation tools such as Playwright to sequentially crawl multiple pages while retaining the cookies and session information set during authentication. It verifies that the session has not expired on each page (e.g., confirming that no elements indicating a logout state are displayed), and if the session has expired, it executes the login process again. This process 203 allows for efficient crawling of authenticated web applications without requiring re-authentication for each page, and enables the collection of metadata for all pages.

The control policy 210 is a declarative rule definition that the determination unit 150 and the execution unit 170 refer to in order to determine whether an operation can be executed, whether confirmation is required, the correction grace period, etc. This policy 210 includes risk classification 211, risk level 212, confirmation requirement 213, correction grace time 214, correction grace period 215, and grace period management 216, and is defined as an external parameter file. Specifically, a risk level is set for each operation type (create, update, delete, etc.), and explicit confirmation (CONFIRM) is applied to high-risk operations, automatic execution after correction grace period is applied to medium-risk operations, and immediate execution or a short grace period is applied to low-risk operations. The length of the correction grace period is variably set based on the risk level of the operation and the reliability of the speech analysis. This policy 210 allows for customization of control rules for each target Web application 20 without changing the algorithm itself.

Risk classification 211 is a definition in control policy 210 that classifies each operation type into three levels: high risk, medium risk, and low risk. This classification 211 is set considering the severity of the operation's impact, its reversibility, and its scope of external impact. Specific classification examples: High risk (deletion, external transmission, settlement, permission change, important data update), Medium risk (addition, update, status change), Low risk (search, list display, transition, explanation request). The determination unit 150 refers to the risk classification 211 and outputs explicit confirmation (CONFIRM) for high-risk operations, allows automatic execution after a correction grace period for medium-risk operations, and applies immediate execution or a short grace period for low-risk operations. This classification 211 minimizes the risk of erroneous operations while maximizing user convenience.

Risk level 212 is a specific numerical or enumerated value for high risk, medium risk, and low risk as defined in risk classification 211 in control policy 210. This level 212 is set for each operation type in an external parameter file. For example, the delete operation has a high risk level, the create operation has a medium risk level, and the retrieve operation has a low risk level. The determination unit 150 and execution unit 170 refer to the risk level 212 to determine whether confirmation is necessary 213 and the correction grace period 214. Depending on the characteristics of the target web application 20, the risk level of specific operations can be adjusted (e.g., in a financial system, the create operation is also treated as high risk). This level 212 enables flexible risk management.

The confirmation requirement 213 is a logical value that defines whether or not explicit confirmation (CONFIRM) from the user 30 is required before each operation is executed in the control policy 210. This requirement 213 is determined based on the risk level 212 and the confidence level of the speech analysis. Specific rule examples: If the risk level is "high," confirmation is always required (CONFIRM); if the risk level is "medium" or "low" and the element selection score is between 0.5 and 0.8, confirmation is required; if the score is 0.8 or higher, confirmation is not required (automatic execution after correction grace period). The determination unit 150 outputs either "CONFIRM" or "EXECUTE" as the next action based on the confirmation requirement 213. This requirement 213 optimizes the balance between safety and convenience.

The correction grace period 214 is a numerical value (in seconds) defined in the control policy 210 that defines the grace period for the user 30 to correct or interrupt their utterance before automatically executing a medium-risk or low-risk operation. This time 214 is variably set based on the risk level of the operation and the confidence level of the speech analysis. Specific setting examples: 0.5 seconds for low-risk operations with high confidence, 1.5 seconds for medium-risk operations or operations with moderate confidence, and 3.0 seconds for medium-risk operations with low confidence. If additional utterances are made during the correction grace period, the timer is reset; if an interruption keyword ("stop," "catch," etc.) is used, execution is canceled; and if "go" is used, execution is performed immediately. This time 214 reduces the risk of misrecognition in voice operations while enabling smooth, continuous operation.

The correction grace period 215 is the time range defined in the control policy 210 for actually managing the period defined in the correction grace time 214. This period 215 refers to the period from the time the utterance is confirmed until the correction grace time 214 has elapsed. During the correction grace period 215, the agent 50 monitors the user 30 for additional utterances or interruption operations. If an additional utterance is detected, the timer is reset to accept continuation or correction of the utterance. If an interruption keyword or UI operation (e.g., clicking the "Cancel" button) is detected, the execution is canceled. If the period has elapsed without interruption, the operation is automatically executed. This period 215 achieves a middle ground between the "confirmation button each time" and "fully automatic execution" of the voice UI, enabling smooth, continuous operation while preventing errors.

The grace period management system 216 is a management mechanism in the control policy 210 that performs timer control, additional utterance detection, interruption keyword detection, and immediate execution keyword detection during the correction grace period 215. This mechanism 216 performs the following processes: (1) Starts the timer when the utterance is confirmed; (2) Continuously monitors the user's voice input while the timer is running; (3) If an additional utterance is detected, resets the timer, overwrites the utterance text, and performs HASP analysis again; (4) If an interruption keyword ("stop," "by," etc.) is detected, immediately stops the timer and cancels the execution; (5) If an immediate execution keyword ("go," "execute," etc.) is detected, immediately executes the operation; (6) If the timer has elapsed without interruption until the correction grace period 214, calls the operation execution mechanism 171. This mechanism 216 enables flexible control of voice operations.

The conversation history 220 is historical information that records past utterances, operation history, selection results, etc., in the dialogue between user 30 and agent 50. This history 220 is managed by the context maintenance mechanism 221 and referenced by the determination unit 150 and the operation target prediction mechanism 230. The conversation history 220 includes the timestamp of each utterance, utterance text, HASP analysis results (5 slots), determination result (next action), execution result (success/failure), user 30's selection result (which of the candidates presented by SUGEST was selected), corrected utterances (utterances before and after correction), etc. If the scope is not explicitly stated from user 30's utterance, the determination unit 150 refers to the conversation history 220 to predict the operation target and sets the prediction source identifier 232 to "Prediction from conversation history". This history 220 enables dialogue with a continuous context, allowing for appropriate interpretation even when user 30 makes abbreviated utterances ("add it," "edit number 3," etc.).

The context maintenance mechanism 221 manages the conversation history 220 and maintains contextual information for the current dialogue session. This mechanism 221 initializes an empty history at the start of the dialogue session, adds utterance information after each utterance, and clears the history at the end of the session (e.g., Widget close, no utterances for a certain period). Contextual information includes the last mentioned scope (e.g., "List of Personnel"), the last manipulated entity (e.g., "Mr. Tanaka"), and transition history (e.g., "Dashboard → List of Personnel → Personnel Details"). The determination unit 150 refers to the context maintenance mechanism 221 and infers the target of the operation from an abbreviated utterance ("edit it"). This mechanism 221 enables dialogue with shared context, similar to natural human conversation, eliminating the need for the user 30 to give complete instructions each time.

The target of operation prediction mechanism 230 is a processing mechanism that predicts the target of operation by referring to past conversation history 220 or operation history 231 when the target of operation is not explicitly stated in the user's utterance. This mechanism 230 makes predictions based on the following rules: (1) It adopts the last mentioned target (e.g., if the user was previously on "List of Personnel," "Add" is predicted to be "Add Personnel"), (2) It adopts the last entity operated on (e.g., if "Tanaka-san" was displayed previously, "Edit" is predicted to be "Edit Tanaka-san"), and (3) It adopts the current page's target as the default. The predicted target of operation is set to "Prediction from Conversation History" in the prediction source identifier 232 and confirmed by the user 30 through confirmation processing 233. This mechanism 230 allows for the handling of abbreviated utterances and maintains a natural flow of dialogue.

The operation history 231 is a record of previously executed operations, referenced by the target entity prediction mechanism 230. This history 231 includes the timestamp of each operation, the operation type (create, update, delete, etc.), the target entity (scope, actor), details of the target entity (e.g., user ID of "Tanaka-san"), and the execution result (success/failure). When user 30 uses demonstrative pronouns such as "edit it" or "delete this," the target entity prediction mechanism 230 identifies the last displayed or operated entity from the operation history 231 and predicts it as the target entity. This history 231 eliminates the need for user 30 to repeatedly utter specific entity names, enabling more natural and efficient dialogue. Furthermore, the operation history 231 is also used for learning successful patterns; similar operations performed frequently are saved as successful patterns.

The inference source identifier 232 is an identifier that indicates the source from which the operation target inferred by the operation target inference mechanism 230 was inferred. This identifier 232 can be set to values such as "explicit specification" (explicitly included in the user 30's utterance), "inference from conversation history" (inferred from past utterances), "inference from operation history" (inferred from past operations), and "inference from current page" (inferred from the currently displayed page). The determination unit 150 refers to the inference source identifier 232 and, if it is "inference from conversation history" or "inference from operation history" and the operation type is a write operation (create, update, delete), it calls the confirmation process 233 to confirm the inferred operation target with the user 30. In the case of "explicit specification," it is executed immediately without confirmation. This identifier 232 enables appropriate control according to the uncertainty of the inference, preventing erroneous operations.

The confirmation process 233 is a process that confirms the operation target predicted by the operation target prediction mechanism 230 with the user 30. This process 233 is executed when the prediction source identifier 232 is "predicted from conversation history" or "predicted from operation history" and the operation type is a write operation (create, update, delete). Specifically, a confirmation message such as "Do you want to add XX?" or "Are you sure you want to edit XX?" is presented to the user 30. If the user 30 gives a positive response such as "Yes" or "OK," the prediction source identifier 232 is updated to "explicitly specified" and the operation is executed. If the

user gives a negative response such as "No" or "Wrong," the operation is canceled and the correct operation target is asked again (ASK). This process 233 prevents erroneous operations based on predictions, while enabling smooth, continuous operations by executing immediately without confirmation in the case of explicit specification.

The operation pattern database 250 is a database that stores successful operation patterns (operation intent, target entity, starting path, target path, operation procedure, field mapping) as vector data. This database 250 records the number of uses and successes of each pattern via confidence update 251, dynamically updating the confidence level. When agent 50 receives a new utterance, it performs a vector search of the operation pattern database 250. If a similar pattern exists with a confidence level above a predetermined threshold, it reuses that pattern without calling the Large-Scale Language Model (LLM). This reduces LLM calls and speeds up response time by 30 to 50 times. The database 250 stores successful patterns, failure-to-success patterns (differences in action plans that succeeded in retries), user selection results (which candidate was selected in SUGEST), and correction patterns (corrected utterances and revised action plans), etc.

The confidence update 251 is a process that records the number of uses and successes for each pattern in the operation pattern database 250 and dynamically updates the confidence level. This process 251 increments the usage count each time a pattern is reused, increments the success count if the operation is successful, and recalculates the confidence level (e.g., confidence level = success count / usage count). Patterns with high confidence levels are reused even with a low similarity threshold during vector search, while patterns with low confidence levels are reused cautiously with a higher similarity threshold. If an operation fails, the failure count is incremented, the confidence level decreases, and patterns exceeding a certain failure rate are removed from the database. This process 251 allows for autonomous learning through use, prioritizing the reuse of highly reliable patterns and continuously improving response speed and accuracy.

The automatic test scenario generation 260 is a process that estimates the operations that can be performed on a page (create, update, delete, retrieve, etc.) from the metadata (5-slot structured data) and form element information of each page discovered during crawling, and automatically generates a test scenario (operation procedure, input data template, expected result) written in natural language. This process 260 compares the existing test scenario with the new crawl results during recrawl via the test scenario update 261, detecting the addition of new pages, modifications to existing pages, and page deletions, and automatically updates the test scenario. The generated test scenario's execution results are recorded by the test scenario result recorder 262 and used as input for self-improvement of declarative decision parameters (claim 15). This process 260 automatically generates operation tests for the target web application 20 in natural language format without manual test case definition, and automatically updates the test scenario through recrawl even when screens are added or modified.

Test scenario update 261 is a process in the automatic test scenario generation 260 that compares existing test scenarios with new crawl results during recrawl and automatically updates the test scenarios. This process 261 detects and responds to the following changes: (1) Addition of new pages: Automatically generates and adds test scenarios for new pages; (2) Changes to existing pages: Detects changes in form structure (addition, deletion, or modification of fields) and updates the operation procedures for the corresponding test scenarios; (3) Deletion of pages: Disables or deletes the test scenarios corresponding to deleted pages. The changes are visualized on the dashboard and can be reviewed and approved by the user 30 (company representative). This process 261 automatically tracks test scenarios in response to continuous changes in the target web application 20, maintaining test comprehensiveness and up-to-dateness, and reducing maintenance costs.

The test scenario result recording 262 is a process that executes the test scenarios generated by the automatic test scenario generation 260 and records the execution results (success/failure, error details in case of failure, execution time, etc.). This process 262 passes each test scenario in natural language format to the input conversion unit 70, performs HASP analysis, judgment, element selection, and DOM operations, and determines success/failure by comparing the expected result (e.g., "XX is displayed in the list") with the actual result. The execution results are stored in the tenant-unit metadata management database 60 and visualized as a test result report on the dashboard. For failed test scenarios, the cause of failure (element not found, operation unexecutable, result differs from expected, etc.) is analyzed and used as a test case for self-improvement of declarative judgment parameters (claim 15). This process 262 realizes self-healing test automation (claim 16), enabling resilience to UI changes and continuous quality assurance.

The speech expression 270 is natural language utterance entered by the user 30 via voice or text. This expression 270 includes a variety of utterance forms, such as operation instructions (e.g., "Register Tanaka as a user"), explanation requests (e.g., "What is a care manager?"), selection responses (e.g., "Select option 2"), correction utterances (e.g., "Actually, add him as the person in charge"), interruption instructions (e.g., "Stop," "Quit"), and immediate execution instructions (e.g., "Go," "Execute"). The speech expression 270 is converted into text by the speech recognition engine. If additional utterances are added during the correction grace period, the text is overwritten. The utterance is finalized when the correction grace period has elapsed. The finalized utterance is converted into a definition expression 271 (5-slot structured data) by the input conversion unit 70, and the next action is determined by the determination unit 150. This expression 270 allows the user 30 to operate the target web application 20 using only natural speech, without complex mouse and keyboard operations.

The definition expression 271 is a 5-slot structured data (Actor, Operation, Scope, Constructs, Context) generated by the input conversion unit 70 by analyzing the utterance expression 270. This expression 271 is structured according to the HASP slot structure 272, and the correspondence with the HASP slots of the current page is analyzed by the slot space projection 273. Each slot contains a slot name (e.g., actor, operation), a slot value (e.g., "Tanaka", create), type information (e.g., actor.type: person), and confidence level (e.g., confidence: 0.9). The determination unit 150 compares the definition representation 271 with the current page's state slots, identifies which slots are missing using the missing slot detection 274, classifies the gap patterns using the gap detection 275, and determines a strategy for filling in the missing slots using the completion 276. This representation 271 achieves both the flexibility of natural language and the rigor of structured data, realizing a highly accurate determination logic.

Slot structure 272 is the structural definition of the five slots in the definition expression 271 and structured metadata 62. This structure 272 consists of the following five slots: (1) Actor: who and what the operation targets (e.g., "Tanaka", actor.type: person), (2) Operation: the intent of the business operation (CRUD) (e.g., create, retrieve, update, delete, explain), (3) Scope: on which page and in which area the operation is performed (e.g., "List of Personnel", "User Registration"), (4) Constraints: conditions associated with the operation (e.g., "10 items", "Date order", "Tokyo only"), (5) Context: Conversational state and execution control state (e.g., confidence, scopeSource, isCorrectionIntent, navigationHistory). This structure 272 is used as the same model for page description during crawling and speech analysis during execution (bilateral HASP), achieving high-precision semantic matching between pages and utterances.

Slot space projection 273 is a process that projects the definition expressions 271 (input slots) extracted from the utterance and the structured metadata 62 (state slots) of the current page onto a common slot space and analyzes the correspondence between them. This process 273 compares the values of the input slot and the state slot for each slot (Actor, Operation, Scope, Constructs, Context) to determine whether they match, mismatch, or are missing. For example, if the input slot's scope is "User" and the state slot's scope is "Person in Charge," the scope is determined to be "mismatched." This determination result is classified as a gap pattern in gap detection 275 and used in completion 276 to determine a strategy for completing missing slots. This process 273 maximizes the advantages of using the same 5-slot model for page description and utterance analysis, achieving highly accurate semantic matching.

The missing slot detection 274 is a process that identifies which slots are missing in the slot space projection 273, either input slots (extracted from utterances) or state slots (current page). For each slot, this process 274 determines whether a value exists (filled), is unspecified (missing), or is unknown (ambiguous). For example, if user 30 utters "Add Tanaka-san," it is determined that actor: Tanaka (filled), operation: create (filled), scope: unknown (missing). The result of the missing slot detection 274 is used by the determination unit 150 when determining the next action. If all three slots (actor, operation, scope) are missing, the system will determine the appropriate response based on the missing information pattern: "ASK (ask for information)"; if only "scope" is missing, it will either "guess from conversation history" or "SUGGEST (suggest a candidate)"; and if all slots are present, it will "EXECUTE (execute the operation)".

Gap detection 275 is a process that integrates the results of slot spatial projection 273 and missing slot detection 274 to classify the gap between input slots and state slots into multiple patterns. This process 275 is the core of the HASP Gap Model (gap-driven decision engine). Specific gap pattern examples: (1) All three slots are missing: "ASK (ask for information)", (2) Scope matches the current page: "EXECUTE (execute operation on the current page)", (3) Scope does not match the current page: "EXECUTE (automatically transition to the corresponding page)", (4) actor.type does not match the current page: "SUGGEST (present candidate pages)", (5) Only operation is specified: "EXECUTE (operate on the current page)". The determination unit 150 maps the result of gap detection 275 to a declarative table and determines the next action (EXECUTE, SUGGEST, ASK, CONFIRM, ANSWER). This process 275 eliminates complex conditional branching and realizes a highly maintainable determination logic based on declarative definitions.

Completion 276 is a process that executes a strategy to complete missing slots based on the gap patterns classified by gap detection 275. This process 276 includes the following completion strategies: (1) Completing the GAP between the utterance scope and the current page → Page transition (EXECUTE), (2) Completing the utterance operation if it is the same as the utterance scope for the current page → Operation execution (EXECUTE), (3) Completing missing utterance scope and operation through dialogue → Question/choice (ASK /SUGGEST), (4) Inferring missing slots from conversation history → Call the operation target inference mechanism 230 and set the inference source identifier 232, (5) Identifying the target from the currently displayed list in Flash RAG → Identifying the target entity by vector search. The determination unit 150 determines the next action based on the result of completion 276, and the execution unit 170 executes the operation. This process 276 appropriately responds to ambiguous and abbreviated utterances from the user 30, enabling smooth dialogue operation.

<Second Embodiment>
The second embodiment is implemented as a browser extension (e.g., Chrome Extension). In this embodiment, each part of the agent is implemented as a content script and background script of the browser extension.
Content script section (operates in the context of the target web page):
- Execution unit: Directly executes DOM operations - Element selection unit: Searches for and identifies elements on the page - Part of the input conversion unit: Accepts user input - Background script unit (runs in the background of the browser extension):

- Input conversion unit main part: semantic analysis by language model - Metadata storage unit: storage of crawled metadata - Decision unit: next action determination - Element information storage unit: storage of semantic feature information

The data flow of this embodiment is as follows:
(1) The user provides natural language input through the extension's UI. (2) The content script sends the input to the background script. (3) The background script performs semantic analysis and judgment processing. (4) The judgment result (ActionPlan) is returned to the content script. (5) The content script performs DOM operations.

<Metadata Management>
The metadata storage unit saves structured metadata to the browser extension's local storage (e.g., chrome.storage.local). Upon initial startup or metadata update, the latest metadata is downloaded from the server and saved locally. This enables basic operation detection even in offline environments.

<Differences from SaaS models>
In Embodiment 1 (SaaS type), it was necessary to embed script tags in the target web application, but in this embodiment, since it operates as a browser extension, no code addition to the target application is required. End users can use the functions of the present invention for any web application simply by installing the extension in their browser.

<Technical Effects>
- No administrator privileges required for the target application - Applicable across multiple web applications - Easy customization to suit individual user environments

<Embodiment 3>
In this embodiment, an existing RPA platform (e.g., UiPath, Blue Prism) is used.
It is implemented as a function of Automation Anywhere, etc. In this embodiment, each part of the agent is implemented as a custom activity or library of the RPA platform.
RPA platform layer:
- Workflow definition engine - Browser control module (existing RPA functionality)
- Custom activity group of the present invention (implementation of the present invention):
- "Natural Language Web Manipulation" Activity - Includes input conversion and judgment units - Receives natural language instructions from the user and generates an ActionPlan - "Semantic Element Selection" Activity
- Implement element selection - Element identification using Semantic Selector - "Get Metadata" activity - Implement acquisition process - Crawl target web application and generate structured metadata

<Example of workflow definition>
In the RPA workflow design screen, it can be defined as follows:

Step 1: Retrieve metadata (first time only)
- TargetURL: https://example. com/admin
- OutputVariable: siteMetadata

Step 2: Launch browser (standard RPA function)
-Browser: Chrome
- URL: https://example. com/admin

Step 3: Natural Language Web Manipulation (Activity of the present invention)
- InputText: Display list of responsible persons - Metadata: siteMetadata
-Browser: currentBrowser

Step 4: Natural Language Web Manipulation (Activity of the present invention)
- InputText: Added Taro Tanaka - Metadata: siteMetadata
-Browser: currentBrowser

<Integration with RPA platform>
The custom activity in this embodiment calls the browser control API of the RPA platform to perform DOM operations. For example, in the case of UiPath, it works in conjunction with the Browser function of UiPath.Core.Activities to perform element clicks, inputs, transitions, etc. via Selenium WebDriver. If the determination unit outputs EXECUTE, the generated ActionPlan (a sequence of operations such as navigate, click, fill, etc.) is converted into standard browser operation commands of the RPA platform and executed.

<Metadata Management>
The metadata storage unit stores structured metadata in the RPA platform's storage (e.g., UiPath's Orchestrator Assets, Blue Prism's Work Queues, etc.). This allows metadata to be shared among multiple RPA robots, enabling unified natural language manipulation.

<Technical Effects>
- Enables integration of natural language manipulation into existing RPA workflows - RPA recording function is unnecessary (defined in natural language)
- Reduce maintenance costs when changing the UI (semantic self-healing)
- Making it easier for non-technical users to create RPA workflows.

<Fourth Embodiment>
The fourth embodiment is implemented as a standalone desktop application and is configured to control the browser, for example, via Chrome DevTools Protocol (CDP) or Selenium WebDriver.

<System Configuration>
In this embodiment, the agent is implemented as a desktop application that integrates all of its functions.
Desktop application layer:
- Main window (implemented using Electron, etc.)
- Voice input UI
- Operation history display - Metadata management UI
- Agent Core - Input Conversion Unit - Metadata Storage Unit - Determination Unit - Element Information Storage Unit - Browser Control Layer - CDP Connection Module - or Selenium WebDriver Connection Module Controlled Browser (Example):
- Chrome (controlled via CDP)
- Firefox (controlled via WebDriver)
- Edge (controlled via CDP)

The operation flow of this embodiment is as follows:

(1) When the desktop application is launched, the user selects a browser. (2) The application connects to the browser using CDP or WebDriver. (3) The user provides natural language input via voice or text. (4) The agent core performs semantic analysis and judgment processing. (5) DOM operations are performed via the browser control layer.

<Example of implementing element selection via CDP>
The element selection unit identifies elements using the following procedure:

(1) Obtain the CSS selector of the target element from the metadata. (2) Execute document.querySelector (selector) with the CDP's Runtime.evaluate command. (3) If the element is not found (UI change),
Runtime.evaluate retrieves all elements,
Perform scoring based on semantic features (4) Obtain the coordinates of the element with the highest score using DOM.getBoxModel (5) Perform a click using Input.dispatchMouseEvent

<Metadata Management>
The metadata storage unit stores structured metadata in the desktop application's local database (e.g., SQLite). It centrally manages metadata from multiple web applications and automatically selects the appropriate metadata when switching between applications. Metadata retrieval can also be performed within this desktop application; when the user specifies a URL to crawl, it autonomously discovers the site structure in the background and generates and saves the metadata.

<Multi-site integrated operation>
A key feature of this embodiment is the ability to perform operations across multiple web applications in an integrated manner.

Example: "Retrieve customer information from CRM and create an invoice in the accounting system."
(1) The judgment unit detects two scopes: "CRM" and "accounting system". (2) Customer information is retrieved via the browser by referring to the metadata of the CRM site. (3) The retrieved information is stored. (4) An invoice is created via the browser by referring to the metadata of the accounting system. (5) The stored customer information is automatically entered.

This functionality is achieved by having the metadata storage unit hold metadata from multiple sites.

<Technical Effects>
- Enables unified operation of multiple web applications within the company - No browser extension installation required - Easy centralized management and distribution by administrators - Improves operational efficiency through multi-site integrated operation - Meets enterprise security requirements

The present invention will be described in more detail below with reference to examples, but the present invention is not limited to these examples.

Figure 1 shows the overall system configuration of the Web application operation support service according to this embodiment. As shown in Figure 1, the self-healing operation support system 10 is a computer-based information processing system that includes the target Web application 20, users 30, scripts 40, agents 50, and a tenant-unit metadata management database 60.

The target web application 20 is an existing business system, and script 40 is embedded within its pages. Script 40 contains a tenant-specific identifier 41, and agent 50 uses this identifier 41 upon startup to retrieve the corresponding metadata from the tenant-specific metadata management database 60.

Agent 50 is software that operates within the user 30's browser and comprises an input conversion unit 70, a metadata storage unit 80, an element information storage unit 90, a determination unit 150, an element selection unit 160, and an execution unit 170. When user 30 provides input in natural language, Agent 50 receives it and autonomously performs operations on the target web application 20.

The tenant-level metadata management database 60 holds structured metadata 62, which includes an operator category 63, an operation type category 64, a target area category 65, a constraint category 66, and a context category 67. Furthermore, a UI change detection mechanism 180 is provided to detect changes in the UI structure of the target web application 20 and update the metadata incrementally.

In the system shown in Figure 1, the structure and operable elements of each page of the target web application 20 are automatically collected by the acquisition process 190 and the authentication process 200, and stored in the tenant-unit metadata management database 60. The agent 50 refers to the metadata obtained from this database 60 to interpret the user 30's input and execute operations on the target web application 20.

Figure 2 is a flowchart showing the processing flow from natural language input by user 30 to operation execution in this embodiment. As shown in Figure 2, the process begins with the authentication process (ST1). For the target web application 20 that requires authentication, the authentication information is temporarily received by the temporary authentication information reception 201, and the login form is automatically identified by the automatic login identification 202 to complete the authentication. After authentication is complete, the authentication information is discarded from memory and is not permanently stored.

Next, in the acquisition process (ST2), the structure and operable elements of each page of the target web application 20 are acquired. This acquisition process 190 determines the target pages to crawl using navigation area analysis 191 and link extraction 192, collects element information including dynamically expanded forms using action button detection 193 and dynamic form recording 194, and collects explanatory information using help text extraction 195 and tooltip extraction 196.

The acquired information is described in structured metadata generation (ST3) as structured metadata 62 consisting of five categories: subject, operation type, target area, constraints, and context. This structured metadata 62 is stored in the tenant-specific metadata management database 60 during database storage (ST4).

In the runtime phase, during user natural language input (ST5), user 30 provides input via voice or text. The input conversion unit 70, in speech analysis (ST6), converts user 30's natural language input into speech expressions 270 and structures them into five categories according to the slot structure 272.

The determination unit 150, in the decision-making process (ST7), compares the utterance expression 270 and the definition expression 271 using the slot space projection 273, and performs missing slot detection 274 and gap detection 275. The detected missing slots or gaps are filled in by the completion 276, and the next action is determined.

In element selection (ST8), the element selection unit 160 identifies the element to be operated on. It attempts a search using location information 91, and if unsuccessful, searches for similar elements using semantic identification features 92. Finally, in operation execution (ST9), the execution unit 170 executes the operation on the target Web application 20 based on the decision of the determination unit 150.

Figure 3 shows the fallback search control mechanism for identifying DOM elements according to this embodiment.
As shown in Figure 3, the element selection unit 160 first attempts to search using location information 91. Location information 91 includes CSS selectors, XPate, data attributes, etc. If this search is successful, the element is identified and processing is completed.

If the search using location information 91 fails, the fallback search control 161 attempts a search using identifier attributes. This search uses the semantic identifier assigned during crawling to find the element. If this search is successful, the element is identified and processing is completed.

If searching by identifier attribute also fails, a similar element search is performed using semantic identification features 92. Semantic identification features 92 include display text, element attribute information, relationships with neighboring elements, and relative position within the form. The similarity of each candidate element is calculated using weighted scoring 162.

In the weighted scoring 162, if the candidate with the highest similarity can be uniquely determined, that element is selected as the target of the operation, and the processing is completed. On the other hand, if multiple candidates have equivalent scores, the candidate presentation mechanism 163 and the user confirmation switching mechanism 164 will either prompt the user 30 for confirmation or present candidates, depending on the risk level of the operation.

This fallback search control allows the target application 20 to be rediscovered based on its semantic identification features 92, even if its UI structure is changed, enabling continuous operation without script modification.

Figure 4 shows the mechanism for saving and reusing successful operation patterns according to this embodiment.
As shown in Figure 4, when the agent 50 receives new natural language input from the user 30, it first searches the operation pattern database 250. In this search, it looks for a pattern similar to the current input from among the operation patterns that have been successful in the past.

If similar patterns exist with a confidence level above a predetermined threshold, the pattern is reused without calling the language model 240. This reduces LLM calls and significantly speeds up response time. On the other hand, if similar patterns are below the threshold, or if no suitable pattern exists, analysis by the language model 240 is performed.

Once an operation plan is generated through pattern reuse or language model analysis, the operation is executed. The result of the operation is determined as success or failure. If the operation is successful, the operation sequence is stored in the operation pattern database 250, and the confidence level is dynamically updated by the confidence update 251 based on the number of uses and successes.

If the operation fails, the system returns to receiving user input and attempts the operation in a different way. This learning loop enables faster and more stable operations for similar utterances in subsequent attempts. Confidence update 251 is performed each time the pattern is used; the more successful the pattern is, the higher its confidence becomes, and the more frequently it is reused.

This embodiment allows for the addition of natural language operation functionality without modifying the existing web application 20, and enables the realization of a web application operation support system 10 that can continue to operate even after UI structure changes. Furthermore, by learning successful patterns, the response speed of frequently occurring operations is improved, providing users 30 with a smoother operation experience.

<Appearance 1>
A method for enabling natural language operation without modifying an existing web application 20, comprising the steps of: acquiring the structure and operable elements of each page of the target web application 20, describing each page as structured metadata 62 consisting of multiple categories including subject, operation type, target area, constraints, and context; extracting and associating semantically identifiable features 92 with each operable element in addition to location information 91; saving the metadata 62 in a referable format; and enabling continuous operation even after a change in the UI structure by having an agent 50 operating in the user 30's browser environment convert the user 30's natural language input into the same format as the structured metadata 62, and when identifying an operable element by referring to the metadata 62, searching for similar elements using semantically identifiable features 92 if the search using location information 91 fails.

According to this embodiment, flexible operation using natural language can be achieved without any modification to existing web applications. By rediscovering the target of operation based not only on location information but also on semantic characteristics, operation

continuity can be maintained even if UI changes or layout differences occur, significantly reducing operational costs and maintenance burden. Furthermore, by describing the page structure using multifaceted categories, it becomes possible to respond to ambiguous user instructions, resulting in improved operational accuracy and user experience.

<Aspect 2>
A Web application operation support method 100 characterized in that, in embodiment 1, the multiple categories consist of five categories: an operator category 63, an operation type category 64, a target area category 65, a constraint category 66, and a context category 67.

According to this embodiment, the accuracy of natural language input interpretation is improved because the operation content can be organized from five clear perspectives. By analyzing the operation subject and context, appropriate processing can be performed according to the situation, even for the same operation word, leading to a reduction in errors. Furthermore, because the category structure is general-purpose, a common analysis framework can be applied across different web applications, facilitating cross-platform operation and expansion.

<Appearance 3>
A Web application operation support method 100 characterized in that, in embodiment 2, the acquisition step includes a step of performing a navigation area analysis 191 within the seed URL page and prioritizing the extraction of links 192 within said area; a step of detecting action buttons 193 on each page, recording forms 194 that are dynamically expanded by the operation of said buttons, and recording semantic features 92 for each input field in the form; and a step of extracting help text 195 and tooltips 196 contained in each page and saving them in association with metadata 62.

According to this embodiment, it is possible to acquire user-specific navigation paths and operation points, generating high-quality metadata that reflects actual operation. By recording dynamic forms and supplementary explanatory information, it becomes possible to respond to input assistance and explanation requests, thereby increasing the comprehensiveness of operation support. As a result, it is possible to achieve advanced interactive operation support, including understanding assistance, rather than just simple screen operations.

<Appearance 4>
The Web application operation support method 100 in embodiment 3 is characterized in that the acquisition step further includes the steps of performing a temporary receipt of authentication information 201 as an authentication process 200 and completing authentication by automatic login identification 202, discarding the authentication information from memory after authentication is completed, and traversing all pages by browser session maintenance traversal 203.

According to this embodiment, structural information can be securely obtained even from applications requiring authentication, thereby reducing security risks. Since authentication information is not persisted, there is no concern about information leakage, making it easily applicable to enterprise systems. Furthermore, by patrolling within the same session, state-dependent pages can be obtained without omission, creating a highly comprehensive user support environment.

<Aspect 5>
The method according to Embodiment 1 further includes the steps of: estimating the operations that can be performed on each page from the metadata 62 and the contents of the element information holding unit 90 of each page that have been acquired; automatically generating test scenarios written in natural language for each estimated operation; saving the generated test scenarios in association with the metadata 62; and automatically executing a test scenario update 261 by comparing existing test scenarios with newly acquired results when reacquiring data, thereby enabling the automatic generation of operation tests for the target Web application 20 in natural language format from the acquired results without manual test case definition.

According to this embodiment, the results of acquiring screen structure data can be directly used as test assets, significantly reducing the manpower required for test design. Since tests are generated in natural language format, they are highly readable and easy for non-technical users to understand. Furthermore, tests are automatically updated when the UI changes, resulting in reduced maintenance burden and the achievement of continuous quality assurance.

<Aspect 6>
A SaaS-type web application operation support method 100 further comprising the steps of: storing and managing metadata 62 in a tenant-unit metadata management database 60 on a tenant identifier 61 basis; issuing a script 40 corresponding to a tenant and embedding it in the page of the target web application 20, and deploying an agent 50 that operates in the user 30's browser; and the steps of the script 40 using the tenant-specific identifier 41 to obtain the latest metadata 62 at startup, and executing a differential update mechanism 181 when a change is detected by the UI change detection mechanism 180.

According to this embodiment, metadata can be managed securely and efficiently even in a multi-tenant environment, and it becomes easy to provide it as SaaS. Since the client side only needs to embed a script for deployment, the initial burden is low, and centralized management on the server side enables immediate updates and improvements.

<Pattern 7>
A Web application operation support method 100 in embodiment 1, characterized in that semantically identifiable features 92 include displayed text, element attribute information, relationship with neighboring elements, and relative position within a form.

According to this embodiment, multifaceted element identification that does not rely on a single piece of information becomes possible, and the occurrence of misrecognition can be suppressed. By combining features that are robust to UI changes, the target of operation can be rediscovered with high accuracy even if positional shifts or wording changes occur, resulting in the effect of achieving highly self-correcting operation support.

<Patent 8>
A Web application operation support method 100 further comprising the steps of: defining a control policy 210 as an external parameter, including an operation risk classification 211, whether or not pre-execution confirmation is required 213, and a correction grace period 214; distributing and applying the control policy 210 to an agent 50; classifying the interpreted operations into risk levels 212, requiring explicit confirmation for high-risk operations, and automatically executing medium- and low-risk operations after a correction grace period 215; setting the correction grace period 215 variably based on the confidence level; and detecting and controlling additional inputs or interruption instructions during the correction grace period.

This embodiment enables operation control that balances convenience and safety. While ensuring human final judgment for critical operations, minor operations can be automated, thereby improving work efficiency. Furthermore, by changing the behavior according to reliability, it enables gradual automation based on usage history.

<Aspect 9>
A self-healing operation support system 10 that enables operation of an existing web application 20 using natural language or voice without modifying the application itself, comprising: an input conversion unit 70 that operates in the user 30's browser environment and converts natural language input into structured data; a metadata storage unit 80 that holds metadata 62 describing each page in a category structure; an element information storage unit 90 that holds location information 91 and semantic identification features 92; a determination unit 150 that determines the next action based on differences; an element selection unit 160 that performs similar element searches; and an execution unit 170 that executes operations.

According to this embodiment, self-contained operational support is realized as a system, enabling flexible operation solely within the client environment. Since continued use is possible through semantic re-search even when the UI changes, a robust operational foundation suitable for long-term operation can be provided.

<Aspect 10>
In embodiment 9, the system 10 is implemented as a script 40 embedded in the page, and the metadata holding unit 80 obtains the latest metadata 62 based on the tenant identifier 61 and requests differential updates when the UI is changed, characterized in that this is a self-healing operation support system 10.

This embodiment allows for extremely easy implementation and minimizes the impact on the existing environment. Centralized metadata updates enable immediate reflection for all users, improving operational efficiency.

<Pattern 11>
In embodiment 9, the self-healing operation support system 10 is characterized in that the element selection unit 160 sequentially performs location information search, identifier search, and similarity search based on semantic features using fallback search control 161, calculates similarity using weighted scoring 162, and switches to either the candidate presentation mechanism 163 or the user confirmation switching mechanism 164.

This embodiment achieves both search efficiency and safety. By processing in stages, from simple matching to advanced estimation, it prevents errors while increasing the success rate of operations.

<Aspect 12>
In embodiment 9, the self-repairing operation support system 10 is characterized in that the input conversion unit 70 analyzes the operation type and explanation request with the same structure, the determination unit 150 switches between operation execution and explanation response, and continuous dialogue is enabled by the conversation history 220 and context maintenance mechanism 221.

According to this embodiment, users can seamlessly transition between operation and explanation, simultaneously supporting their understanding and execution. This enhances the sophistication of the interactive UI and reduces the learning curve.

<Verification 13>
Embodiment 9 is a self-healing operation support system 10 characterized by saving successful operation analysis results in an operation pattern database 250 and reusing them based on a reliability update 251.

According to this embodiment, processing efficiency and accuracy improve with repeated use, resulting in improved response speed and savings of computing resources.

<Pattern 14>
In embodiment 9, the self-healing operation support system 10 is characterized in that the determination unit 150 infers the operation target based on the conversation history 220 or operation history 231 and performs a confirmation process 233.

This embodiment allows for flexible handling of ambiguous instructions and enables natural conversational operation. Furthermore, security is ensured by performing confirmation during writing operations.

<Pattern 15>
In embodiment 9, a self-healing operation support system 10 is characterized by controlling the judgment process based on external parameters and automatically generating failure analysis and adjustment suggestions.

According to this embodiment, behavioral improvements can be made without program modifications, enabling continuous optimization during operation.

<Aspect 16>
A test automation method 110 using the system 10 described in embodiment 9, characterized in that it structures and transforms natural language test cases, selects elements based on semantic features, and performs operations and records results.

According to this embodiment, testing can continue even after UI changes, significantly improving the reliability and efficiency of regression testing.

<Pattern 17>
A program for causing a computer to function as a component of the self-healing operation support system 10 described in aspect 9.

According to this embodiment, the system can be easily implemented on a general-purpose computer and provided in a form that facilitates distribution and updates.

<Pattern A1>
An information processing method for operating an existing web application based on natural language input,
(A) Visibility range extraction process for the currently displayed page of a web application, which extracts page content within the range visible to the user.
(B) Candidate extraction process that extracts multiple candidates from dynamic data including data rows of a table or list extracted by the visible range extraction process.
(C) A transient vectorization process that vectorizes multiple candidates in memory at the time of the request, discards them after the search is complete, and does not persist them.
(D) Similarity search process that calculates the similarity between a search query corresponding to natural language input and multiple vectorized candidates, ranks the candidates, and identifies the target for operation.
(E) An execution control process that performs a DOM operation on the target without performing additional analysis by a large-scale language model if the ranking result satisfies predetermined conditions.
Information processing methods including

According to this embodiment, it is possible to perform a step-by-step process of narrowing down the target from within the display page of an existing web application based on natural language input. By processing candidate information transiently and avoiding persistence, a configuration that enables high-speed response is achieved while reducing the cost and risk of data retention. Furthermore, since operations can be performed immediately without going through a large-scale language model only when predetermined conditions are met, it is possible to efficiently utilize computing resources and improve overall response performance.

<Pattern A2>
The information processing method described in embodiment A1,
Transient vectorization is an information processing method that enables searches to always reflect the latest DOM state by re-extracting candidates from the DOM state of the currently displayed page in response to each natural language input, vectorizing them, discarding them after the search is complete, and then vectorizing them again in response to the next natural language input.

According to this embodiment, even when the DOM of a web page changes dynamically, it is always possible to extract and search information that reflects the latest state at that moment. This improves the accuracy of identifying the target of the operation and allows for flexible handling of dynamic content. Furthermore, by reconstructing the data each time without retaining past data, both information freshness and security are ensured.

<Pattern A3>
An information processing method described in embodiment A1 or A2,
The execution control process is based on the similarity of the ranked candidates.
(a) If the execution threshold is exceeded, the DOM operation is automatically executed.
(b) If the value is above a predetermined confirmation threshold but below an execution threshold, the user is asked to confirm whether to perform the DOM operation.
(c) An information processing method that requests the user to input additional information if the value is below a verification threshold.

According to this embodiment, the ability to dynamically determine whether an operation can be automatically executed based on similarity can be achieved, thereby reducing the risk of erroneous operation while enabling control that encourages appropriate user intervention. This results in an interface that balances safety and automation without compromising the user experience.

<Pattern A4>
The information processing method described in aspect A3,
An information processing method that, if a DOM operation falls under the category of a high-risk operation including external transmission, registration, update, or deletion, does not automatically execute the DOM operation even if the similarity is above the execution threshold, and instead requests confirmation from the user.

According to this embodiment, even if a high similarity is calculated, automatic execution of operations such as sending or modifying user data is suppressed, and user confirmation is made mandatory, thereby ensuring maximum security. This is a useful mechanism for preventing serious consequences due to malfunctions.

<Pattern A5>
An information processing method described in any one of the descriptions A1 to A4,
(a) If the DOM operation is successful, a success pattern including the operation intent, target entity, start path, target path, operation procedure, and field mapping is stored in the memory unit.
(b) An information processing method that, for subsequent natural language inputs, searches for successful patterns, and when a successful pattern is found under predetermined conditions, skips calling a large-scale language model and executes a DOM operation.

According to this embodiment, patterns of previously successful operations are learned and memorized, enabling rapid responses to similar inputs. This significantly reduces processing time and avoids redundant processing, leading to an overall improvement in system performance.

<Pattern A6>
The information processing method described in aspect A5,
The success pattern includes the semantic characteristics of the elements used to identify the target of the operation. If the DOM operation fails or the element is not found, the system searches for similar elements based on these semantic characteristics, enabling continuous operation even after a change in the UI structure.

According to this embodiment, even if the UI structure of a web page changes, continuity of operation and self-healing can be achieved by re-searching for candidates based on semantic features. This improves the ability to adapt to environmental changes and significantly improves the maintainability of the system.

<Pattern A7>
An information processing method according to any one of the embodiments A1 to A6, wherein page description information obtained by a visible range extraction process and operation instruction information based on natural language input are converted into a common structured format including subject, operation type, target area, constraints, and context, and used for similarity search processing and DOM operation determination.

According to this embodiment, by unifying the information obtained from the visible range and the natural language operation instructions into a common format, it becomes possible to perform search processing and operation decisions with high accuracy and consistency. Standardization of the data structure facilitates smoother cooperation between different components, improving the overall reliability of the system.

(Technical fields of this embodiment A1 to A7)
The present invention relates to an information processing method for automating user operations in existing web applications based on natural language input, and a system for executing the same, and more particularly to a high-speed and secure processing method for identifying, searching for, and executing DOM operations on dynamic content being displayed.

(Background technology)
Traditionally, automating web application operations has widely relied on standardized operation flows and pre-configured, fixed selector information. However, in order to cope with the increase in dynamic content, the high frequency of UI changes, and the emergence of diverse operation patterns, these conventional technologies have faced challenges such as decreased accuracy in identifying the target of operation, lack of maintainability, and high-cost selector management.

(Summary of this embodiment)
In view of the above-mentioned problems, embodiments A1 to A7 provide an information processing method that dynamically extracts candidates from the visible range of a web page and automatically identifies the target of operation based on similarity search with natural language input. Furthermore, by processing these candidates as transient vectors in memory, the search process is executed quickly and safely, and the configuration enables the storage and reuse of successful patterns, as well as the search for similar elements in response to UI changes.

(Summary of Embodiments A1 to A7 in this model)
This embodiment includes an information processing device and method for receiving natural language input from a user on a web application, dynamically identifying an operation target corresponding to that input, and executing an appropriate DOM operation. The information processing device comprises an input processing means, a visible range extraction means, a candidate extraction means, a vectorization processing means, a similarity search means, an execution control means, a success pattern storage means, and a similar element search means.

(Specific processing configurations for embodiments A1 to A7)
As shown in Figure 1, the information processing device has an input processing unit that receives natural language input from the user. When the input processing unit receives a natural language instruction entered as a string, it performs grammatical analysis and preprocessing to extract the user's intent. Next, the visible range extraction unit extracts DOM information within the user's visible range, targeting the web page currently displayed in the browser. The extraction targets include the currently displayed table rows, list items, form fields, etc.

The candidate extraction unit extracts multiple candidate entities contained within dynamic data structures (tables, lists, etc.) from the DOM information obtained by the visible range extraction unit. The extracted candidates, along with their attributes and text content, are stored in temporary memory. The vectorization unit then converts the extracted candidates and natural language input into vector representations. Vectorization uses an embedding method based on a distributed representation model to represent the semantic features of the extracted candidates as numerical vectors.

These vectorizations are processed transiently in memory and discarded after the search is complete, eliminating the need for large-scale persistent storage and enabling high-speed processing without retaining highly sensitive data. The similarity search unit then calculates the similarity between the natural language input vector and the candidate vectors and ranks them. Similarity is measured using metrics such as cosine similarity and Euclidean distance.

The execution control unit performs the following processing branches based on the similarity score. If the similarity score is above a predetermined execution threshold, it automatically executes DOM operations (click, value input, submission, etc.) on the corresponding candidate. If the similarity score is below the execution threshold but above the confirmation threshold, it requests confirmation from the user and executes the operation after obtaining the user's consent. Furthermore, if the similarity score is below the confirmation threshold, it controls the system to request additional information from the user.

Furthermore, if a DOM operation falls under high-risk operations such as external transmission, registration, updating, and deletion, a safety control is implemented that requires user confirmation regardless of the degree of similarity, and prevents automatic execution.

(Memorizing and reusing successful patterns)
The success pattern storage unit stores a success pattern, including the intent of the operation, the target entity, the starting DOM path, the target DOM path, and the field mapping, when a DOM operation is successful. For subsequent natural language inputs, the system first performs a similarity evaluation using the success pattern as the search target. If a pattern meeting predetermined conditions is found, the system skips calling the large-scale language model and directly executes the DOM operation based on that success pattern.

(Searching for similar elements in response to UI changes)
The similar element search unit 108 has the function of dynamically searching for similar elements different from the original candidates by the candidate search unit 103, based on semantic features within the successful pattern, when the UI structure is changed. This enables a configuration that allows continuous operation using semantic features even when the DOM structure of the web page changes.

(effect)
According to this embodiment, web operations based on user natural language input can be automated quickly and securely. In-memory vector processing and a success pattern reuse mechanism significantly reduce processing time compared to conventional technologies, and further improve flexibility in response to UI changes.

(Variant)
The vectorization method is not limited to this embodiment, and other embedding models or hybrid similarity measures can be used. Furthermore, the similarity threshold can be dynamically adjusted and automatically learned based on the user's operation history and environment settings.

100 Web application operation support service provision method 110 Test automation method 120 Decision-making method 10 Self-healing operation support system 20 Target Web application 30 User 40 Script 41 Tenant-specific identifier 50 Agent 60 Tenant-unit metadata management database 61 Tenant identifier 62 Structured metadata 63 Operation subject category 64 Operation type category 65 Target area category 66 Constraint category 67 Context category 70 Input conversion unit 80 Metadata storage unit 90 Element information storage unit 91 Location information 92 Semantic identification features 150 Judgment unit 160 Element selection unit 161 Fallback search control 162 Weighted scoring 163 Candidate presentation mechanism 164 User confirmation switching mechanism 170 Execution unit 171 Operation execution mechanism 172 Explanation response mechanism 180 UI change detection mechanism 181 Differential update mechanism 190 Acquisition process 191 Navigation area analysis 192 Link extraction 193 Action button detection 194 Dynamic form recording 195 Help text extraction 196 Tooltip extraction 200 Authentication processing 201 Temporary receipt of authentication information 202 Automatic login identification 203 Browser session maintenance patrol 210 Control policy 211 Risk classification 212 Risk level 213 Confirmation required 214 Correction grace period 215 Correction grace period 216 Grace period management 220 Conversation history 221 Context maintenance mechanism 230 Operation target inference mechanism 231 Operation history 232 Inference source identifier 233 Confirmation processing 240 Language model 250 Operation pattern database 251 Confidence level update 260 Automatic test scenario generation 261 Test scenario update 262 Test scenario result recording 270 Utterance representation 271 Definition representation 272 Slot structure 273 Slot space projection 274 Missing slot detection 275 Gap detection 276 Completion

Document Claims

Language: (English)

A method for enabling natural language operation on existing web applications run by computers without modifying them,
The computer obtains the structure and manipulable elements of each page of the target web application, and describes each page as structured metadata consisting of multiple categories including subject, operation type, target area, constraints, and context.
The computer performs the steps of extracting and associating semantically identifiable features for each operable element, in addition to location information.
The computer saves the metadata in a format that can be referenced,
A processing means operating in the user's browser environment converts the user's natural language input into the same format as the structured metadata, and when identifying the target element by referring to the metadata, if a search using location information fails, it searches for similar elements using the semantically identifiable features, thereby enabling continuous operation even after a change in the UI structure.
A method for supporting the operation of a web application, characterized by including the following.

In claim 1,
A method for supporting the operation of a web application, characterized in that the aforementioned multiple categories consist of five categories: Actor, Operation, Scope, Constraints, and Context.

In claim 2,
The aforementioned acquisition step is,
The computer analyzes the navigation area within the seed URL page and extracts the links within that area as priority targets.
The computer includes the steps of: detecting action buttons on each page, detecting forms that are dynamically expanded when such buttons are clicked, and recording semantic features for each input field in the form, including label text, attribute information, and neighboring text.
The computer extracts the help text, tooltips, and explanatory text contained in each page and stores them in association with the metadata.
A method for supporting the operation of a web application, characterized by including the following.

In claim 3,
The aforementioned acquisition step is,
The computer, in response to a web application requiring authentication, temporarily receives authentication information at the start of the acquisition process, automatically identifies the login form, and completes the authentication.
The computer, after completing authentication, discards the authentication information from memory and does not permanently store it;
The computer then navigates through all pages while maintaining the same browser session.
A method for supporting the operation of a web application, further comprising the following:

In the method according to claim 1,
The computer then performs the steps of estimating the operations that can be performed on each page from the metadata and element information of each page obtained,
The computer automatically generates test scenarios described in natural language for each of the estimated operations,
The computer saves the generated test scenario in association with the metadata,
The computer compares the existing test scenario with the newly acquired results during reacquisition and automatically updates the test scenario.
It further includes,
A web application operation support method characterized by automatically generating operation tests for a target web application in natural language format from acquired results, without the need for manual definition of test cases.

In claim 1,
The computer stores and manages the metadata in a database on a tenant-by-tenant basis.
The computer issues a script corresponding to the tenant and embeds it within the page of the target web application, thereby deploying an agent that operates within the user's browser.
The script includes an identifier unique to the tenant, and the agent, upon startup, uses the identifier to retrieve the latest metadata corresponding to the tenant from the server, and if it detects a change in the UI of the target application, it performs a differential update by re-retrieving the data.
A method for supporting the operation of a web application, further comprising the following:

In claim 1,
A method for assisting the operation of a web application, characterized in that the semantically identifiable features include display text, attribute information of an element, relationships with neighboring elements, and relative positions within a form.

In claim 1,
The computer defines a control policy as an external parameter, which includes the risk classification of the operation, whether or not pre-execution verification is required, and the grace period for correction.
The computer distributes and applies the control policy to the processing means, thereby reserving the user with final execution rights in update operations.
The processing means, in applying the control policy, includes the steps of classifying the interpreted operations into risk levels, requiring explicit confirmation for high-risk operations, and assigning medium- and low-risk operations to be automatically executed after a correction grace period,
The processing means includes the step of variably setting the length of the correction grace period based on the risk level of the operation and the reliability of the analysis,
The processing means includes the steps of: resetting the grace period if additional input is detected during the correction grace period; canceling execution if an interruption instruction is detected; and automatically executing the operation if the grace period has elapsed without interruption;
A method for supporting the operation of a web application, further comprising the following:

A self-healing operation support system,
An input conversion unit that converts the user's natural language input into structured data,
A metadata storage unit that holds metadata describing the page structure and operable elements of the target web application,
The aforementioned operable element includes an element information holding unit that holds positional information and semantically identifiable features,
A determination unit that determines the next action based on the structured data and metadata,
If the search using the aforementioned location information fails, an element selection unit rediscovers the target element based on the semantically identifiable features,
An execution unit that performs an operation on the target element identified by the element selection unit,
A self-healing operation support system characterized by comprising the following features.

In claim 9,
The aforementioned system is implemented as a script embedded within a page of the target web application.
The metadata storage unit retrieves the latest metadata from the server based on the tenant identifier.
A self-healing operation support system characterized by requesting a differential update from the server when it detects a change in the UI of the target application.

In claim 9,
The element selection unit is,
Search by location information,
Searching by identifier attribute assigned to an element,
Searching for similar elements based on the aforementioned semantically identifiable features,
The following will be executed in this fallback order:
Similarity is calculated by weighted scoring of the aforementioned semantically identifiable features.
A self-healing operation support system characterized by switching between user confirmation and candidate presentation depending on the risk level of the operation when multiple candidates have equivalent scores.

In claim 9,
The aforementioned input conversion unit is
As user utterance operation types, data manipulation (create, retrieve, update, delete) and explanation requests (explain) are analyzed using the same 5-slot structured data format.
The determination unit,
For data manipulation, the system outputs a result indicating whether to perform a DOM operation.
In response to a request for explanation, the system searches for knowledge information associated with the metadata storage unit and outputs a determination result that provides the answer.
The aforementioned system,
A self-healing operation support system characterized by maintaining conversation history in a continuous dialogue where data manipulation and explanation requests are mixed, and enabling switching between operations and explanations while maintaining context.

In claim 9,
An operation pattern database that stores patterns of previously successful operations as vector data,
A means for searching the operation pattern database for new user input and, if a similar pattern exists with a confidence level above a predetermined threshold, for reusing the similar pattern without calling the language model,
A means for performing operational analysis using a language model when the number of similar patterns is below a predetermined threshold, or when no suitable pattern exists.
If the operation analysis by the language model is successful, means for saving the operation pattern based on the results of the operation analysis to a database,
A means for recording the number of uses and successes of the aforementioned operation pattern and dynamically updating the reliability level,
A self-healing operation support system characterized by comprising the following features.

In claim 9,
The determination unit,
If the target of the operation is not explicitly stated from the user's input, a means of inferring the target of the operation by referring to past conversation history or operation history is provided.
Means for setting an identifier indicating the source of the inference for the inferred target of operation,
When the aforementioned identifier indicates an inference from the conversation history and the operation type is a write operation, a means for confirming the inferred operation target with the user,
Equipped with,
If the user confirms, update the identifier to an explicit specification and execute the operation.
A self-healing operation support system characterized by immediately executing without confirmation for inputs where the target of the operation is explicitly specified.

In claim 9,
The determination process is executed based on an external parameter file that controls the operation of the determination unit.
A means to analyze the cause of failure when the judgment result differs from the expected result, based on the execution results of the test cases.
A means for automatically generating suggestions for adjusting the external parameters according to the failure pattern,
The aforementioned adjustment proposal includes means to include an explanation of the side effects of the adjustment,
Means for automatically reflecting the aforementioned adjustment proposal in the external parameter file based on user approval or when predetermined safety conditions are met,
Furthermore,
A self-healing operation support system characterized by its ability to improve the judgment logic solely by adjusting external parameters, without modifying the program itself.

A method for automating the testing of a web application using the system described in claim 9,
The computer converts test cases written in natural language into structured data using the input conversion unit,
The computer, using the element selection unit, identifies the element to be operated on based on the semantically identifiable features rather than location information;
The computer performs an operation on the identified element and records the test results.
Includes,
A test automation method characterized in that the computer can continue to execute tests without redefining test cases, even after the UI of the target web application has been changed, by rediscovering elements based on the semantically identifiable features.

A program for causing a computer to function as a component of the self-healing operation support system described in claim 9.

In claim 9,
The determination unit,
The structured data derived from user input and the metadata of the current page are both stored as multiple slots corresponding to multiple categories including subject, operation type, target area, constraints, and context.
A self-healing operation support system characterized by detecting the difference as a slot-level gap by determining whether each of the multiple slots matches, does not match, or is missing.

In claim 18,
The determination unit,

The detected types or combinations of gaps are classified as gap patterns.
A self-healing operation support system characterized by determining the next action based on the action type associated with the gap pattern.

In claim 19,
The correspondence between the gap pattern and the action type is stored in an external parameter file as a declaratively defined correspondence table, rather than as a conditional branching logic.
A self-healing operation support system characterized in that the determination logic of the determination unit can be adjusted without changing the program itself by changing the external parameter file.

Language: (Japanese)

コンピュータが実行する、既存のＷｅｂアプリケーションを改修することなく自然言語による操作を実現する方法であって、
コンピュータが、対象Ｗｅｂアプリケーションの各ページの構造及び操作可能要素を取得し、各ページを主体、操作種別、対象領域、制約条件、及びコンテキストを含む複数のカテゴリからなる構造化メタデータとして記述するステップと、
前記コンピュータが、各操作可能要素について、位置情報に加えて、当該要素を意味的に識別可能な特徴を抽出して関連付けるステップと、
前記コンピュータが、前記メタデータを参照可能な形式で保存するステップと、
ユーザーのブラウザ環境において動作する処理手段が、ユーザーの自然言語入力を前記構造化メタデータと同一形式に変換し、前記メタデータを参照して操作対象要素を特定する際、位置情報による検索が失敗した場合に、前記意味的に識別可能な特徴を用いて類似要素を探索することで、ＵＩ構造変更後も継続的に動作可能とするステップと、
を含むことを特徴とするＷｅｂアプリケーション操作支援方法。

請求項１において、
前記複数のカテゴリは、操作主体（Ａｃｔｏｒ）、操作種別（Ｏｐｅｒａｔｉｏｎ）、対象領域（Ｓｃｏｐｅ）、制約条件（Ｃｏｎｓｔｒａｉｎｔｓ）、及びコンテキスト（Ｃｏｎｔｅｘｔ）からなる５つのカテゴリである
ことを特徴とするＷｅｂアプリケーション操作支援方法。

請求項２において、
前記取得のステップは、
前記コンピュータが、シードＵＲＬのページ内のナビゲーション領域を解析し、当該領域内のリンク先を優先対象として抽出するステップと、
前記コンピュータが、各ページにおいてアクションボタンを検出し、当該ボタンをクリックして動的に展開されるフォームを検出し、
フォーム内の各入力フィールドについて、ラベルテキスト、属性情報、及び近傍テキストを含む意味的特徴を記録するステップと、
前記コンピュータが、各ページに含まれるヘルプテキスト、ツールチップ、及び説明文を抽出し、前記メタデータと関連付けて保存するステップと、
を含むことを特徴とするＷｅｂアプリケーション操作支援方法。

請求項３において、
前記取得のステップは、
前記コンピュータが、認証が必要なＷｅｂアプリケーションに対して、取得開始時に認証情報を一時的に受け取り、ログインフォームを自動識別して認証を完了するステップと、
前記コンピュータが、認証完了後は当該認証情報をメモリから破棄し永続保存しないステップと、
前記コンピュータが、同一のブラウザセッションを維持しながら全ページを巡回するステップと、
をさらに含むことを特徴とするＷｅｂアプリケーション操作支援方法。

請求項１に記載の方法において、
前記コンピュータが、取得した各ページのメタデータ及び要素情報から、当該ページで実行可能な操作を推定するステップと、
前記コンピュータが、推定した各操作について、自然言語で記述されたテストシナリオを自動生成するステップと、
前記コンピュータが、生成したテストシナリオを前記メタデータと関連付けて保存するステップと、
前記コンピュータが、再取得時に既存テストシナリオと新規取得結果を比較し、テストシナリオを自動更新するステップと、
をさらに含み、
手動によるテストケース定義なしに、取得結果から対象Ｗｅｂアプリケーションの操作テストを自然言語形式で自動生成する
ことを特徴とするＷｅｂアプリケーション操作支援方法。

請求項１において、
前記コンピュータが、前記メタデータをテナント単位でデータベースに蓄積し管理するステップと、
前記コンピュータが、前記テナントに対応するスクリプトを発行し、対象Ｗｅｂアプリケーションのページ内に埋め込むことで、ユーザーのブラウザ内で動作するエージェントを配置するステップと、
前記スクリプトは、前記テナントに固有の識別子を含み、前記エージェントが起動時に前記識別子を用いて、前記テナントに対応する最新のメタデータをサーバーから取得し、対象アプリケーションのＵＩ変更を検出した場合に、再取得により差分更新を行うステップと、
をさらに含むことを特徴とするＷｅｂアプリケーション操作支援方法。

請求項１において、
前記意味的に識別可能な特徴は、表示テキスト、要素の属性情報、近傍要素との関係、及びフォーム内の相対位置を含む
ことを特徴とするＷｅｂアプリケーション操作支援方法。

請求項１において、
前記コンピュータが、操作のリスク分類、実行前の確認要否、及び訂正猶予時間を含む制御ポリシーを外部パラメータとして定義するステップと、
前記コンピュータが、前記制御ポリシーを前記処理手段に配布して適用することで、更新系操作における最終実行権限をユーザーに留保するステップと、
前記処理手段が、前記制御ポリシーの適用において、解釈された操作をリスクレベルに分類し、高リスク操作については明示的確認を要求し、中・低リスク操作については訂正猶予期間を経て自動実行するよう振り分けるステップと、
前記処理手段が、前記訂正猶予期間の長さを、操作のリスクレベル及び解析の信頼度に基づいて可変設定するステップと、
前記処理手段が、前記訂正猶予期間中において、追加入力を検出した場合は猶予期間をリセットし、中断指示を検出した場合は実行をキャンセルし、猶予期間が中断なく経過した場合に操作を自動実行するステップと、
をさらに含むことを特徴とするＷｅｂアプリケーション操作支援方法。

自己修復型操作支援システムであって、
ユーザーの自然言語入力を構造化データに変換する入力変換部と、
対象Ｗｅｂアプリケーションのページ構造及び操作可能要素を記述したメタデータを保持するメタデータ保持部と、
前記操作可能要素について、位置情報及び意味的に識別可能な特徴を保持する要素情報保持部と、
前記構造化データ及び前記メタデータに基づいて次アクションを判定する判定部と、
前記位置情報による検索が失敗した場合に、前記意味的に識別可能な特徴に基づいて操作対象要素を再発見する要素選択部と、
前記要素選択部により特定された操作対象要素に対して操作を実行する実行部と、
を備えることを特徴とする自己修復型操作支援システム。

請求項９において、
前記システムは、対象Ｗｅｂアプリケーションのページ内に埋め込まれたスクリプトとして実装され、
前記メタデータ保持部は、テナント識別子に基づいてサーバーから最新のメタデータを取得し、
対象アプリケーションのＵＩ変更を検出した場合に、サーバーに対して差分更新を要求する
ことを特徴とする自己修復型操作支援システム。

請求項９において、
前記要素選択部は、
位置情報による検索と、
要素に付与された識別子属性による検索と、
前記意味的に識別可能な特徴による類似要素探索と、
をこの順序でフォールバック実行し、
前記意味的に識別可能な特徴の重み付きスコアリングにより類似度を算出し、
複数の候補が同等のスコアを持つ場合は、操作のリスクレベルに応じてユーザー確認または候補提示に切り替える
ことを特徴とする自己修復型操作支援システム。

請求項９において、
前記入力変換部は、
ユーザー発話の操作種別として、データ操作（ｃｒｅａｔｅ、ｒｅｔｒｉｅｖｅ、ｕｐｄａｔｅ、ｄｅｌｅｔｅ）と説明要求（ｅｘｐｌａｉｎ）を同一の５スロット構造化データ形式で解析し、
前記判定部は、
データ操作に対してはＤＯＭ操作を実行する判定結果を出力し、
説明要求に対しては前記メタデータ保持部に関連付けられたナレッジ情報を検索して回答する判定結果を出力し、
前記システムは、

データ操作と説明要求が混在する連続的な対話において会話履歴を維持し、文脈を継続したまま操作と説明を切り替え可能とする
ことを特徴とする自己修復型操作支援システム。

請求項９において、
過去に成功した操作のパターンをベクトルデータとして記憶する操作パターンデータベースと、
新たなユーザー入力に対して前記操作パターンデータベースを検索し、類似パターンが所定の閾値以上の信頼度で存在する場合に、言語モデルを呼び出すことなく当該類似パターンを再利用する手段と、
類似パターンが所定の閾値未満である場合、または適切なパターンが存在しない場合に、言語モデルによる操作解析を実行する手段と、
言語モデルによる操作解析の結果が成功した場合、当該操作解析の結果に基づく操作パターンをデータベースに保存する手段と、
前記操作パターンの使用回数及び成功回数を記録し、信頼度を動的に更新する手段と、
を備えることを特徴とする自己修復型操作支援システム。

請求項９において、
前記判定部は、
ユーザーの入力から操作対象が明示されていない場合、過去の会話履歴または操作履歴を参照して操作対象を推測する手段と、
前記推測された操作対象に推測元を示す識別子を設定する手段と、
前記識別子が会話履歴からの推測を示し、かつ操作種別が書き込み操作である場合に、推測した操作対象をユーザーに確認する手段と、
を備え、
ユーザーが確認した場合は前記識別子を明示的指定に更新して操作を実行し、
操作対象が明示的に指定されている入力については確認なしに即時実行する
ことを特徴とする自己修復型操作支援システム。

請求項９において、
前記判定部の動作を制御する外部パラメータファイルに基づいて判定処理を実行し、
テストケースの実行結果に基づいて、判定結果が期待と異なる場合に失敗原因を分析する手段と、
失敗パターンに応じて、前記外部パラメータの調整提案を自動生成する手段と、
前記調整提案には、調整による副作用の説明を含める手段と、
前記調整提案を、ユーザーの承認に基づいて、または所定の安全条件を満たす場合に自動的に、前記外部パラメータファイルに反映する手段と、
をさらに備え、
プログラム本体を変更することなく、外部パラメータの調整のみで判定ロジックを改善可能とする
ことを特徴とする自己修復型操作支援システム。

請求項９に記載のシステムを用いたＷｅｂアプリケーションのテスト自動化方法であって、
コンピュータが、自然言語で記述されたテストケースを、前記入力変換部により構造化データに変換するステップと、
前記コンピュータが、前記要素選択部により、位置情報ではなく前記意味的に識別可能な特徴に基づいて操作対象要素を特定するステップと、
前記コンピュータが、特定した要素に対して操作を実行し、テスト結果を記録するステップと、
を含み、
前記コンピュータが、対象ＷｅｂアプリケーションのＵＩ変更後も、前記意味的に識別可能な特徴に基づく要素再発見により、テストケースの再定義なしにテストを継続実行可能とする
ことを特徴とするテスト自動化方法。

コンピュータを、請求項９に記載の自己修復型操作支援システムの各部として機能させるためのプログラム。

請求項９において、
前記判定部は、
ユーザーの入力由来の構造化データと現在ページのメタデータとを、いずれも主体、操作種別、対象領域、制約条件及びコンテキストを含む複数のカテゴリに対応する複数スロットとして保持し、
前記複数スロットの各々について、一致、不一致又は欠損を判定することにより、前記差異をスロット単位のギャップとして検出する
ことを特徴とする自己修復型操作支援システム。

請求項１８において、
前記判定部は、
検出した前記ギャップの種類又は組合せをギャップパターンとして分類し、
当該ギャップパターンに対応付けられたアクション種別に基づいて前記次アクションを決定する
ことを特徴とする自己修復型操作支援システム。

請求項１９において、
前記ギャップパターンと前記アクション種別との対応関係が、条件分岐ロジックではなく、宣言的に定義された対応表として外部パラメータファイルに保持され、
前記外部パラメータファイルを変更することにより、プログラム本体を変更することなく前記判定部の判定ロジックを調整可能である
ことを特徴とする自己修復型操作支援システム。